

3. Transformes

Transformers are a neural network architecture proposed in the paper “Attention Is All You Need” (Vaswani et al., 2017). They revolutionized natural language processing (NLP) and later found applications in other fields as well—computer vision, audio, biology, and time series analysis.

The operating principle of a Transformer is based on the attention mechanism, which allows the model to dynamically focus on the most relevant parts of the input data, instead of processing it sequentially like RNNs or LSTMs.

3.1 Transofrmers in NLP

3.1.1 General concept

Preprocessing

- Tokenization
- Converting sentences into sequences of integer numbers (words ids
- Padding

I love this movie! $\xrightarrow{\text{Preprocessing}}$ $[\text{CLS}], id_I, id_{love}, id_{this}, id_{movie}, [\text{SEP}]$

```
from tensorflow.keras import layers

pos_phrases = [
    "i love this movie", "what a great day", "amazing product highly recommend",
    "fantastic results very happy", "pleasant surprise exceeded expectations",
    "brilliant idea well executed", "superb quality", "wonderful experience",
    "happy with the outcome", "this is excellent"
]
neg_phrases = [
    "i hate this movie", "what a terrible day", "awful product do not buy",
    "disappointing results very unhappy", "bad idea poorly executed",
```

```

    "horrible quality", "worst experience", "sad with the outcome",
    "this is terrible", "completely unacceptable"
]

texts = pos_phrases + neg_phrases

max_len = 8
vectorizer = layers.TextVectorization(max_tokens=1000,
output_sequence_length=max_len)
vectorizer.adapt(texts)

vocab_size = len(vectorizer.get_vocabulary())
print("Vocabulary size:", vocab_size)

print(vectorizer.get_vocabulary())

Vocabulary size: 50
['', '[UNK]', np.str_('this'), np.str_('with'), np.str_('what'),
np.str_('very'), np.str_('the'), np.str_('terrible'),
np.str_('results'), np.str_('quality'), np.str_('product'),
np.str_('outcome'), np.str_('movie'), np.str_('is'), np.str_('idea'),
np.str_('i'), np.str_('happy'), np.str_('experience'),
np.str_('executed'), np.str_('day'), np.str_('a'), np.str_('worst'),
np.str_('wonderful'), np.str_('well'), np.str_('unhappy'),
np.str_('unacceptable'), np.str_('surprise'), np.str_('superb'),
np.str_('sad'), np.str_('recommend'), np.str_('poorly'),
np.str_('pleasant'), np.str_('not'), np.str_('love'),
np.str_('horrible'), np.str_('highly'), np.str_('hate'),
np.str_('great'), np.str_('fantastic'), np.str_('expectations'),
np.str_('excellent'), np.str_('exceeded'), np.str_('do'),
np.str_('disappointing'), np.str_('completely'), np.str_('buy'),
np.str_('brilliant'), np.str_('bad'), np.str_('awful'),
np.str_('amazing')]

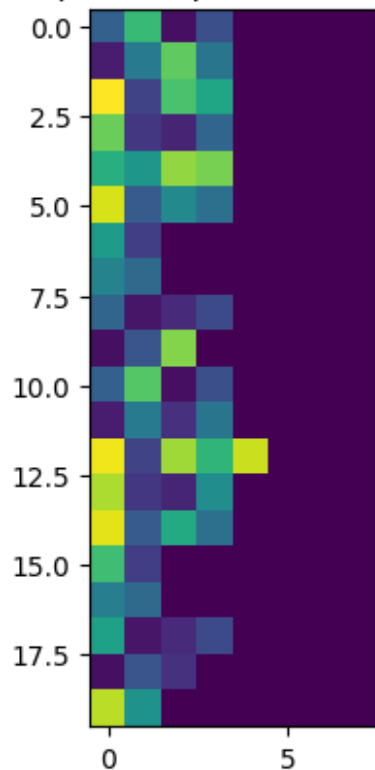
X = vectorizer(texts)
X.shape

TensorShape([20, 8])

import matplotlib.pyplot as plt
plt.imshow(X)
plt.title(f'{X.shape[0]} sequences padded to length: {X.shape[1]}')
plt.show()

```

20 sequences padded to length: 8



```
# for PyTorch

import torch

def build_vocab(texts):
    vocab = {"<PAD>": 0, "<UNK>": 1}
    for t in texts:
        for w in t.lower().split():
            if w not in vocab:
                vocab[w] = len(vocab)
    return vocab

def encode(text, vocab, max_len=8):
    tokens = [vocab.get(w, 1) for w in text.lower().split()]
    tokens = tokens[:max_len]
    return tokens + [0]*(max_len - len(tokens))

vocab = build_vocab(texts)

print(f"vocabulary size: {len(vocab)}")
print(vocab)

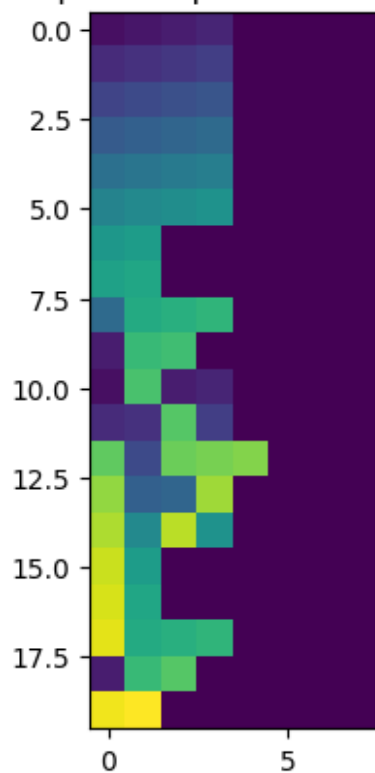
max_len = 8
X = torch.tensor([encode(t, vocab, max_len) for t in texts])
```

```
plt.imshow(X.detach().numpy())
plt.title(f'{X.shape[0]} sequences padded to length: {X.shape[1]}')
plt.show()
```

vocabulary size: 50

```
{'<PAD>': 0, '<UNK>': 1, 'i': 2, 'love': 3, 'this': 4, 'movie': 5,
'what': 6, 'a': 7, 'great': 8, 'day': 9, 'amazing': 10, 'product': 11,
'highly': 12, 'recommend': 13, 'fantastic': 14, 'results': 15, 'very':
16, 'happy': 17, 'pleasant': 18, 'surprise': 19, 'exceeded': 20,
'expectations': 21, 'brilliant': 22, 'idea': 23, 'well': 24,
'executed': 25, 'superb': 26, 'quality': 27, 'wonderful': 28,
'experience': 29, 'with': 30, 'the': 31, 'outcome': 32, 'is': 33,
'excellent': 34, 'hate': 35, 'terrible': 36, 'awful': 37, 'do': 38,
'not': 39, 'buy': 40, 'disappointing': 41, 'unhappy': 42, 'bad': 43,
'poorly': 44, 'horrible': 45, 'worst': 46, 'sad': 47, 'completely':
48, 'unacceptable': 49}
```

20 sequences padded to length: 8



Embedding and positional encoding

Each token is represented as embedding vector (of selected length). Vectors are arranged into matrices.

$$\left[[\text{CLS}], t_1, t_2, \dots, t_n, [\text{SEP}] \right] \xrightarrow{\text{Embedding Layer}} X = [x_{\text{CLS}}, x_1, \dots, x_n, x_{\text{SEP}}]$$

$$\begin{array}{c} + \text{Positional Encoding} \\ \rightarrow \quad Z = X + PE \end{array}$$

While embeddings store representation of token values, positional encoding gives information on their position within the sentence (document).

$$PE_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

where d_{model} is the embedding size, pos - a position within the sequence.

```
import numpy as np
import matplotlib.pyplot as plt

def positional_encoding(seq_len, d_model):
    positions = np.arange(seq_len)[:, np.newaxis]          # shape
    (seq_len, 1)
    dimensions = np.arange(d_model)[np.newaxis, :]        # shape (1,
    d_model)
    angle_rates = 1 / np.power(10000, (2 * (dimensions // 2)) /
    np.float32(d_model))
    angle_rads = positions * angle_rates

    # Apply sin to even indices, cos to odd indices
    pos_encoding = np.zeros_like(angle_rads)
    pos_encoding[:, 0::2] = np.sin(angle_rads[:, 0::2])
    pos_encoding[:, 1::2] = np.cos(angle_rads[:, 1::2])

    return pos_encoding

# Example parameters
seq_len = 100
d_model = 8

pos_encoding = positional_encoding(seq_len, d_model)

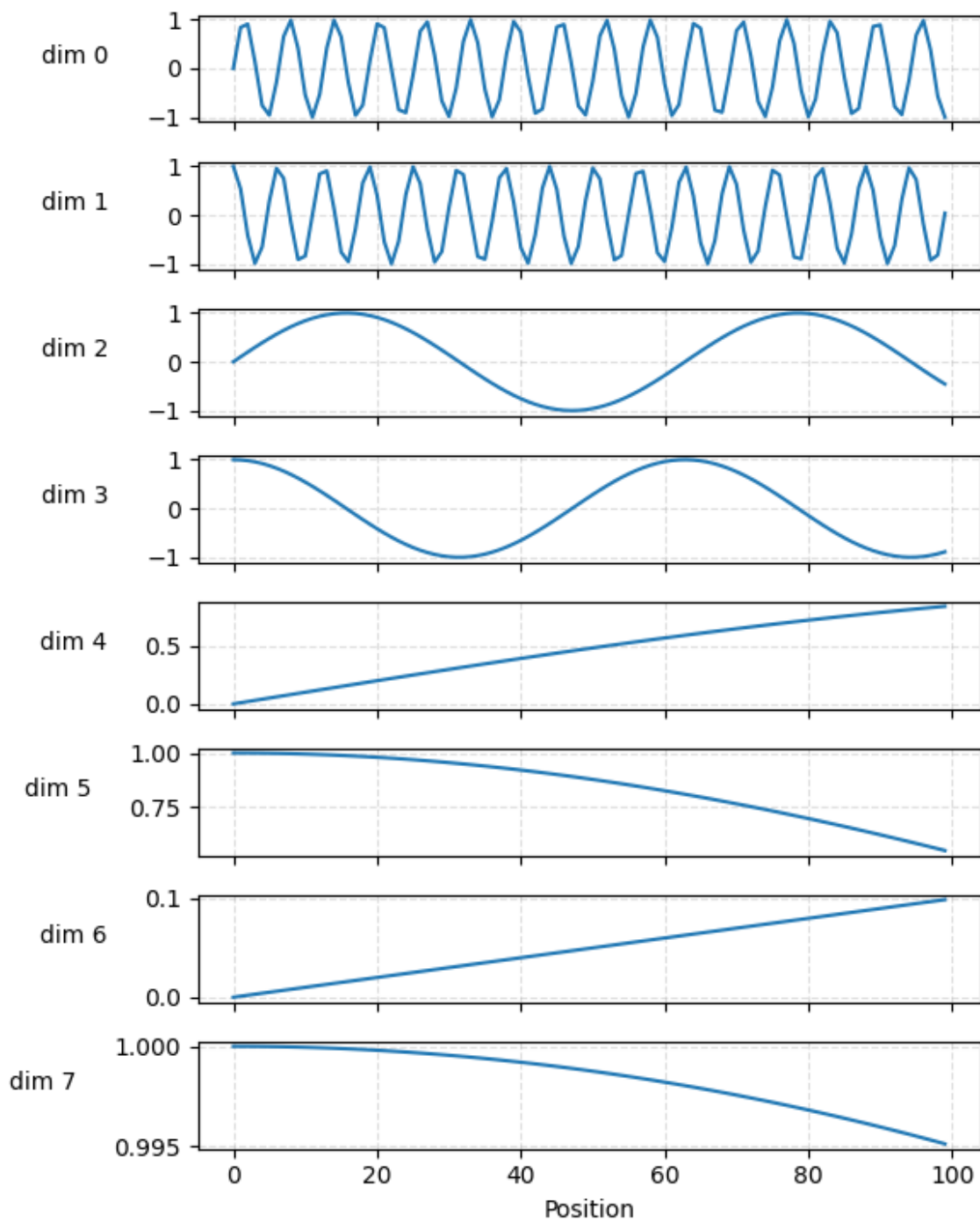
# Plot
fig, axes = plt.subplots(d_model, 1, figsize=(6, d_model),
sharex=True)

for i, ax in enumerate(axes):
    ax.plot(pos_encoding[:, i])
    ax.set_ylabel(f"dim {i}", rotation=0, labelpad=30, fontsize=10)
    ax.grid(True, linestyle="--", alpha=0.4)

axes[-1].set_xlabel("Position")
fig.suptitle(f"Sinusoidal Positional Encodings (d_model={d_model})",
fontsize=14)
```

```
plt.tight_layout(rect=[0, 0, 1, 0.97])  
plt.show()
```

Sinusoidal Positional Encodings (d_model=8)



Distances between positional encodings of neighboring positions

```

import numpy as np
import matplotlib.pyplot as plt

def positional_encoding(seq_len, d_model):
    positions = np.arange(seq_len)[: , np.newaxis] # (seq_len, 1)
    i = np.arange(d_model)[np.newaxis, :] # (1, d_model)
    angle_rates = 1 / np.power(10000, (2 * (i // 2)) /
np.float32(d_model))
    angle_rads = positions * angle_rates

    pos_encoding = np.zeros_like(angle_rads)
    pos_encoding[:, 0::2] = np.sin(angle_rads[:, 0::2])
    pos_encoding[:, 1::2] = np.cos(angle_rads[:, 1::2])
    return pos_encoding

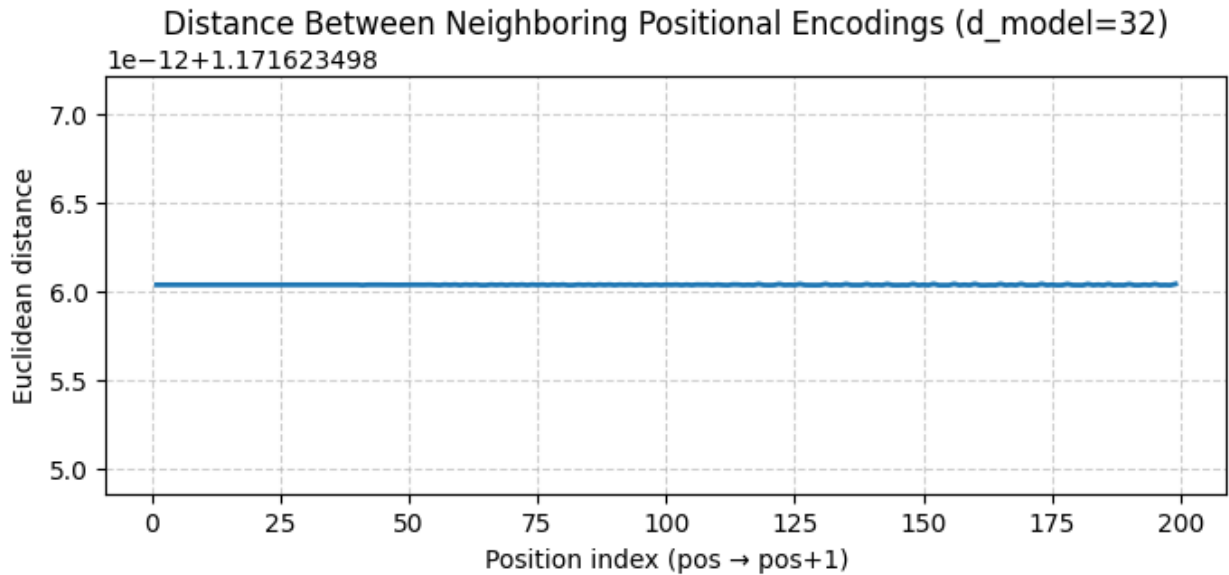
# Parameters
seq_len = 200
d_model = 32

# Compute positional encodings
pos_encoding = positional_encoding(seq_len, d_model)

# Compute Euclidean distances between neighboring encodings
diffs = np.diff(pos_encoding, axis=0)
distances = np.linalg.norm(diffs, axis=1)

# Plot
plt.figure(figsize=(8, 3))
plt.plot(np.arange(1, seq_len), distances, lw=2)
plt.title(f"Distance Between Neighboring Positional Encodings
(d_model={d_model})")
plt.xlabel("Position index (pos → pos+1)")
plt.ylabel("Euclidean distance")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

```



- Neighboring positions are not identical, but they're close — roughly 1.1-1.3 units apart in a high-dimensional space.
- This gives the model a smooth positional gradient, while still keeping positions distinct.

Transformer block

In the next step $Z = X + P E$ (a matrix, which columns are sums of embeddings and positional encoding) is supplied to the transformer block.

$$\begin{array}{c}
 \text{Transformer Encoder Stack } (\times N) \\
 Z \quad \rightarrow \quad \left(\begin{array}{l}
 Q = Z W_Q, K = Z W_K, V = Z W_V \\
 \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \\
 \text{Add \& LayerNorm} \\
 \text{FeedForward: } \text{ReLU}(x W_1 + b_1) W_2 + b_2 \\
 \text{Add \& LayerNorm}
 \end{array} \right)
 \end{array}$$

The most important element is the *Attention* layer. For each position in a sequence (e.g., each word in a sentence), a Transformer computes three vectors:

- $Q = Z W_Q$ - query: *What am I trying to attend to?*
- $K = Z W_K$ - key: *What does this element represent?*
- $V = Z W_V$ - value: *What information do I carry?*

The attention score between two elements is computed as the dot product of the Query and the Key:

$$score(i,j)=Q_i \cdot K_j$$

This score determines how much element i should pay attention to element j . After applying softmax, these scores are used to compute a weighted sum of the Values V , producing the output of the attention mechanism.

Multi-head attention is an extension of the standard attention mechanism in Transformers. Instead of computing a single attention map, the model uses multiple “heads,” each with its own set of Query, Key, and Value projections. Each head can focus on different aspects of the sequence simultaneously — for example, one head might capture syntactic relationships while another captures semantic dependencies.

Attention map example

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import seaborn as sns

# === Input data ===
vocab = {"<PAD>":0, "<UNK>":1, "i":2, "love":3, "this":4, "movie":5}
max_len = 4

def encode(text):
    tokens = [vocab.get(w, 1) for w in text.lower().split()]
    tokens = tokens[:max_len]
    return tokens + [0]*(max_len - len(tokens))

sentence = "i love this movie"
words = sentence.split()

x = torch.tensor([encode(sentence)], dtype=torch.long) # shape: (1, seq_len)

# === Embeddings (random) ===
embed_dim = 8
seq_len = x.shape[1]
vocab_size = 10
embed_dim = 128
num_heads = 1

embedding = nn.Embedding(vocab_size, embed_dim)
x = torch.arange(seq_len) % vocab_size # [0,1,2,3]
x = embedding(x).unsqueeze(1) # (seq_len, batch, embed_dim)

# === Multi-head attention ===
multihead_attn = nn.MultiheadAttention(embed_dim=embed_dim,
num_heads=num_heads, batch_first=False)

# Self-attention: query, key, value = x
```

```

attn_output, attn_weights = multihead_attn(x, x, x, need_weights=True,
average_attn_weights=False)

print("Attention weights shape:", attn_weights.shape)
# -> (num_heads, batch, seq_len, seq_len)

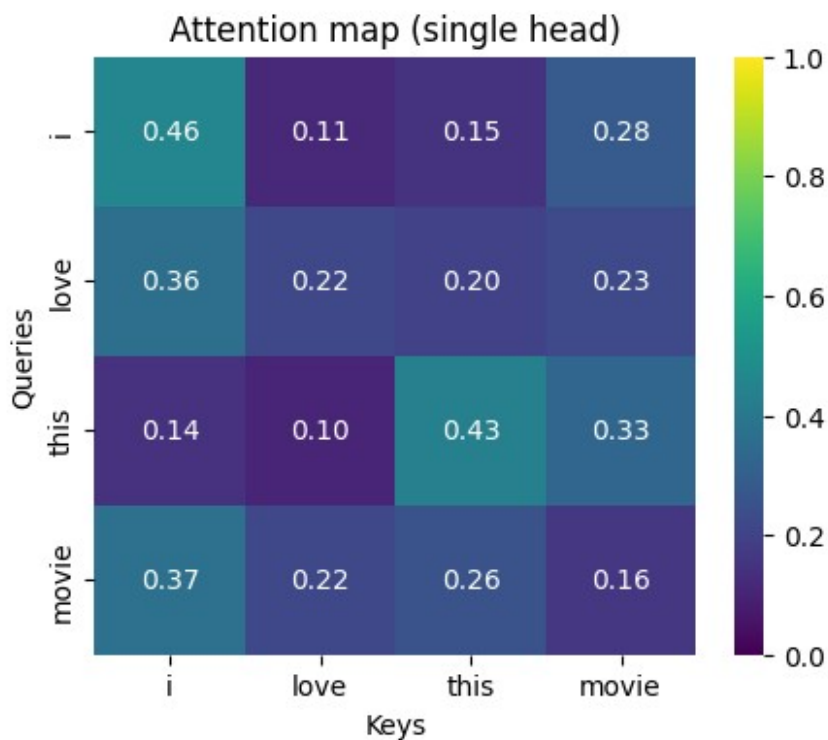
# === Visualization of attention map ===
attn_maps =
attn_weights.squeeze(1).detach().numpy().reshape(num_heads,4,4) #
(num_heads, seq_len, seq_len)

if num_heads==1:
    fig, ax = plt.subplots(figsize=(5, 4))
    sns.heatmap(attn_maps[0], vmin=0, vmax=1, cmap="viridis",
                xticklabels=words, yticklabels=words,annot=True,
fmt=".2f", ax=ax)
    ax.set_title("Attention map (single head)")
    ax.set_xlabel("Keys")
    ax.set_ylabel("Queries")
else:
    fig, axes = plt.subplots(1, num_heads, figsize=(4*num_heads+1, 4))
    for i, ax in enumerate(axes):
        sns.heatmap(attn_maps[i], vmin=0, vmax=1, cmap="viridis",
                    xticklabels=words, yticklabels=words,annot=True,
fmt=".2f", ax=ax)
        ax.set_title(f"Head {i+1}")
        ax.set_xlabel("Keys")
        ax.set_ylabel("Queries")

    plt.tight_layout()
plt.show()

Attention weights shape: torch.Size([1, 1, 4, 4])

```



Classification

Transformer block returns a sequence of hidden states:

$$H_{\text{enc}} = [h_{\text{CLS}}, h_1, \dots, h_n, h_{\text{SEP}}]$$

Depending on the architecture:

- Only the first vector h_{CLS} is further used in classification.
- Or classification is based on all or averaged values of $h_1, \dots, h_n$

The next steps (classification):

$$\square \rightarrow H_{\text{enc}} = [h_{\text{CLS}}, h_1, \dots, h_n, h_{\text{SEP}}]$$

$$\text{Take } h_{\text{CLS}} \rightarrow h_{\text{out}} = h_{\text{CLS}} \in R^{d_{\text{model}}}$$

or

$$\text{Take } h_1, \dots, h_n \rightarrow h_{\text{out}} = \text{Mean}(h_1, \dots, h_n)$$

Classification Head (Dense Layer)

$$\rightarrow \text{logits} = h_{\text{out}} W_c + b_c$$

Softmax

$$\rightarrow P(y | X) = \text{softmax}(\text{logits})$$

$$\begin{array}{c} \text{Loss (categorical crossentropy)} \\ \rightarrow \end{array} L = - \sum_{c=1}^C 1(y=c) \log P(y_c | X)$$

3.1.2 Tiny example

Tensorflow

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# --- Dane ---
pos_phrases = [
    "i love this movie", "what a great day", "amazing product highly recommend",
    "fantastic results very happy", "pleasant surprise exceeded expectations",
    "brilliant idea well executed", "superb quality", "wonderful experience",
    "happy with the outcome", "this is excellent"
]
neg_phrases = [
    "i hate this movie", "what a terrible day", "awful product do not buy",
    "disappointing results very unhappy", "bad idea poorly executed",
    "horrible quality", "worst experience", "sad with the outcome",
    "this is terrible", "completely unacceptable"
]

texts = pos_phrases + neg_phrases
labels = [1]*len(pos_phrases) + [0]*len(neg_phrases)

# --- Tokenizacja ---
max_len = 8
vectorizer = layers.TextVectorization(max_tokens=1000,
output_sequence_length=max_len)
vectorizer.adapt(texts)

vocab_size = len(vectorizer.get_vocabulary())
print("Vocabulary size:", vocab_size)

# --- Funkcja positional encoding ---
def positional_encoding(max_len, d_model):
    positions = np.arange(max_len)[:, np.newaxis]
    dims = np.arange(d_model)[np.newaxis, :]
    angle_rates = 1 / np.power(10000, (2 * (dims // 2)) /
np.float32(d_model))
    angle_rads = positions * angle_rates
```

```

pos_encoding = np.zeros_like(angle_rads)
pos_encoding[:, 0::2] = np.sin(angle_rads[:, 0::2])
pos_encoding[:, 1::2] = np.cos(angle_rads[:, 1::2])
return tf.cast(pos_encoding[np.newaxis, ...], dtype=tf.float32)

# --- Blok Transformer (1 warstwa) ---
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim)
        self.ffn = keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim)
        ])
        self.norm1 = layers.LayerNormalization(epsilon=1e-6)
        self.norm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs, training=None):
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.norm1(inputs + attn_output)
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.norm2(out1 + ffn_output)

# --- Model ---
embed_dim = 32
num_heads = 2
ff_dim = 64

inputs = keras.Input(shape=(max_len,), dtype=tf.int64)
x = layers.Embedding(vocab_size, embed_dim)(inputs)

# Dodanie positional encoding
pos_encoding = positional_encoding(max_len, embed_dim)
x = x + pos_encoding

# Blok Transformer
x = TransformerBlock(embed_dim, num_heads, ff_dim)(x)
x = layers.GlobalAveragePooling1D()(x) # no class token
x = layers.Dropout(0.1)(x)
x = layers.Dense(32, activation="relu")(x)
x = layers.Dense(1, activation="sigmoid")(x)

```

```

model = keras.Model(inputs, x)
model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])

# --- Trening ---
X = vectorizer(texts)
y = tf.constant(labels, dtype=tf.float32)

history = model.fit(X, y, epochs=35, batch_size=4, verbose=0)

# --- Test ---
test_phrases = [
    "i really love this product",
    "this is horrible",
    "fantastic quality and great idea",
    "disappointing and bad",
    "wonderful surprise",
    "completely bad experience"
]

test_X = vectorizer(test_phrases)
preds = model.predict(test_X)

for phrase, p in zip(test_phrases, preds):
    print(f"{phrase:45s} → sentiment: {'positive' if p[0]>0.5 else
'negative'} ({p[0]:.2f})")

Vocabulary size: 50
1/1 _____ 0s 142ms/step
i really love this product          → sentiment: negative
(0.49)
this is horrible                    → sentiment: negative
(0.43)
fantastic quality and great idea    → sentiment: positive
(0.56)
disappointing and bad               → sentiment: negative
(0.32)
wonderful surprise                  → sentiment: positive
(0.60)
completely bad experience           → sentiment: negative
(0.29)

```

PyTorch

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn import functional as F

# --- Dane ---

```

```

pos_phrases = [
    "i love this movie", "what a great day", "amazing product highly
    recommend",
    "fantastic results very happy", "pleasant surprise exceeded
    expectations",
    "brilliant idea well executed", "superb quality", "wonderful
    experience",
    "happy with the outcome", "this is excellent"
]
neg_phrases = [
    "i hate this movie", "what a terrible day", "awful product do not
    buy",
    "disappointing results very unhappy", "bad idea poorly executed",
    "horrible quality", "worst experience", "sad with the outcome",
    "this is terrible", "completely unacceptable"
]

texts = pos_phrases + neg_phrases
labels = torch.tensor([1]*len(pos_phrases) + [0]*len(neg_phrases),
dtype=torch.float32)

# --- Prosta tokenizacja ---
from collections import defaultdict

def build_vocab(texts):
    vocab = {"<PAD>": 0, "<UNK>": 1}
    for t in texts:
        for w in t.lower().split():
            if w not in vocab:
                vocab[w] = len(vocab)
    return vocab

def encode(text, vocab, max_len=8):
    tokens = [vocab.get(w, 1) for w in text.lower().split()]
    tokens = tokens[:max_len]
    return tokens + [0]*(max_len - len(tokens))

vocab = build_vocab(texts)
max_len = 8
X = torch.tensor([encode(t, vocab, max_len) for t in texts])

vocab_size = len(vocab)
embed_dim = 32
num_heads = 2
ff_dim = 64

# --- Positional Encoding ---
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()

```

```

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
torch.log(torch.tensor(10000.0)) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0) # shape: (1, max_len, d_model)
        self.register_buffer('pe', pe)

    def forward(self, x):
        return x + self.pe[:, :x.size(1), :]

# --- Transformer model ---
class MiniTransformer(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_heads, ff_dim,
max_len):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.pos_encoding = PositionalEncoding(embed_dim, max_len)
        encoder_layer = nn.TransformerEncoderLayer(d_model=embed_dim,
nhead=num_heads, dim_feedforward=ff_dim, batch_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=1)
        self.fc1 = nn.Linear(embed_dim, 32)
        self.fc2 = nn.Linear(32, 1)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos_encoding(x)
        x = self.transformer(x)
        x = x.mean(dim=1) # Global average pooling (no class token)
        x = F.relu(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        return x.squeeze(-1)

# --- Training ---
model = MiniTransformer(vocab_size, embed_dim, num_heads, ff_dim,
max_len)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.BCELoss()

for epoch in range(50):
    optimizer.zero_grad()
    preds = model(X)
    loss = criterion(preds, labels)
    loss.backward()
    optimizer.step()
    if (epoch+1) % 10 == 0:
        acc = ((preds > 0.5).float() == labels).float().mean()

```

```

        print(f"Epoch {epoch+1:02d} | Loss: {loss.item():.4f} | Acc: {acc.item():.2f}")
# --- Test ---
test_phrases = [
    "i really love this product",
    "this is horrible",
    "fantastic quality and great idea",
    "disappointing and bad"
]

X_test = torch.tensor([encode(t, vocab, max_len) for t in test_phrases])
preds = model(X_test).detach()

for t, p in zip(test_phrases, preds):
    print(f"{t:45s} → sentiment: {'positive' if p>0.5 else 'negative'} ({p.item():.2f})")

Epoch 10 | Loss: 0.6796 | Acc: 0.65
Epoch 20 | Loss: 0.6468 | Acc: 0.85
Epoch 30 | Loss: 0.5733 | Acc: 0.85
Epoch 40 | Loss: 0.3930 | Acc: 0.95
Epoch 50 | Loss: 0.1672 | Acc: 1.00
i really love this product → sentiment: positive (0.69)
this is horrible → sentiment: negative (0.06)
fantastic quality and great idea → sentiment: positive (0.80)
disappointing and bad → sentiment: negative (0.07)

```

3.1.3 IMDB Reviews

```

import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

# --- Load dataset ---
max_words = 10000 # vocabulary size
max_len = 200 # max review length

(X_train, y_train), (X_test, y_test) =
imdb.load_data(num_words=max_words)
X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>
17464789/17464789 ————— 0s 0us/step

```
import tensorflow_datasets as tfds
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# --- Load raw IMDB text dataset ---
ds_train = tfds.load('imdb_reviews', split='train',
as_supervised=True)
ds_test = tfds.load('imdb_reviews', split='test', as_supervised=True)

# Convert to Python lists
train_texts, train_labels = [], []
for text, label in tfds.as_numpy(ds_train):
    train_texts.append(text.decode("utf-8"))
    train_labels.append(label)

test_texts, test_labels = [], []
for text, label in tfds.as_numpy(ds_test):
    test_texts.append(text.decode("utf-8"))
    test_labels.append(label)

print(f"Loaded {len(train_texts)} training and {len(test_texts)} test
samples.")
```

WARNING:absl:Variant folder
/root/tensorflow_datasets/imdb_reviews/plain_text/1.0.0 has no
dataset_info.json

Downloading and preparing dataset Unknown size (download: Unknown
size, generated: Unknown size, total: Unknown size) to
/root/tensorflow_datasets/imdb_reviews/plain_text/1.0.0...

```
{"model_id": "d47979e4024f456a8aa18b81276e4ce0", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "32750dafc07f42529679be8933585731", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "df995c815a2244e29c057b1148100693", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "e1c16ee02cb64a2c8d64be9fb5eca3e7", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "d23a0e35b4e449a18a848d0024c74d31", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "499beeb089034d869e716baf53065866", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1562750a67be4d6ab72a3c3f98ec4bce", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "3a75b35641944be8be27d77dab0ef15c", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "9ec396d2c6534e208b803fef0514ccbc", "version_major": 2, "version_minor": 0}
```

Dataset imdb_reviews downloaded and prepared to
/root/tensorflow_datasets/imdb_reviews/plain_text/1.0.0. Subsequent
calls will reuse this data.
Loaded 25000 training and 25000 test samples.

```
from tensorflow.keras.preprocessing.text import Tokenizer  
from tensorflow.keras.preprocessing.sequence import pad_sequences  
import numpy as np
```

```
max_words = 10000    # keep top 10k words  
max_len = 200        # truncate/pad reviews to 200 tokens
```

```
# --- Fit tokenizer on training texts ---  
tokenizer = Tokenizer(num_words=max_words, oov_token="<UNK>")  
tokenizer.fit_on_texts(train_texts)
```

```
# --- Convert to sequences ---  
X_train = tokenizer.texts_to_sequences(train_texts)  
X_test = tokenizer.texts_to_sequences(test_texts)
```

```
# --- Pad sequences ---  
X_train = pad_sequences(X_train, maxlen=max_len, padding='post',  
truncating='post')  
X_test = pad_sequences(X_test, maxlen=max_len, padding='post',  
truncating='post')
```

```
y_train = np.array(train_labels)  
y_test = np.array(test_labels)
```

```
print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")
```

```
Train shape: (25000, 200), Test shape: (25000, 200)
```

TF model

```
from tensorflow.keras import layers, models
```

```
embed_dim = 12  
num_heads = 1
```

```

ff_dim = 64 # feed forward dimension

inputs = layers.Input(shape=(max_len,))
x = layers.Embedding(max_words, embed_dim)(inputs)

# Add a simple Transformer encoder block
x = layers.MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)(x, x)
x = layers.GlobalAveragePooling1D()(x)
x = layers.Dense(ff_dim, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation='sigmoid')(x)

model_tf = models.Model(inputs, outputs)
model_tf.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

model_tf.summary()
hist = model_tf.fit(X_train, y_train, epochs=10, batch_size=512,
validation_split=0.2)

```

Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
input_layer_2 (InputLayer)	(None, 200)	0	-
embedding_1 input_layer_2[0]... (Embedding)	(None, 200, 12)	120,000	
multi_head_attenti... embedding_1[0][0... (MultiHeadAttentio... embedding_1[0][0]	(None, 200, 12)	624	
global_average_poo... multi_head_atten... (GlobalAveragePool...	(None, 12)	0	

dense_4 (Dense)	(None, 64)	832	
global_average_p...			
dropout_5 (Dropout)	(None, 64)	0	dense_4[0][0]
dense_5 (Dense)	(None, 1)	65	dropout_5[0]

Total params: 121,521 (474.69 KB)

Trainable params: 121,521 (474.69 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

40/40 ————— 15s 337ms/step - accuracy: 0.5257 - loss: 0.6925 - val_accuracy: 0.7122 - val_loss: 0.6831

Epoch 2/10

40/40 ————— 22s 386ms/step - accuracy: 0.6841 - loss: 0.6709 - val_accuracy: 0.7654 - val_loss: 0.5869

Epoch 3/10

40/40 ————— 18s 328ms/step - accuracy: 0.7813 - loss: 0.5337 - val_accuracy: 0.8222 - val_loss: 0.3998

Epoch 4/10

40/40 ————— 16s 391ms/step - accuracy: 0.8422 - loss: 0.3703 - val_accuracy: 0.8418 - val_loss: 0.3614

Epoch 5/10

40/40 ————— 14s 345ms/step - accuracy: 0.8733 - loss: 0.3092 - val_accuracy: 0.8656 - val_loss: 0.3305

Epoch 6/10

40/40 ————— 14s 347ms/step - accuracy: 0.8898 - loss: 0.2764 - val_accuracy: 0.8734 - val_loss: 0.3189

Epoch 7/10

40/40 ————— 14s 349ms/step - accuracy: 0.9046 - loss: 0.2469 - val_accuracy: 0.8672 - val_loss: 0.3311

Epoch 8/10

40/40 ————— 20s 334ms/step - accuracy: 0.9129 - loss: 0.2297 - val_accuracy: 0.8260 - val_loss: 0.4369

Epoch 9/10

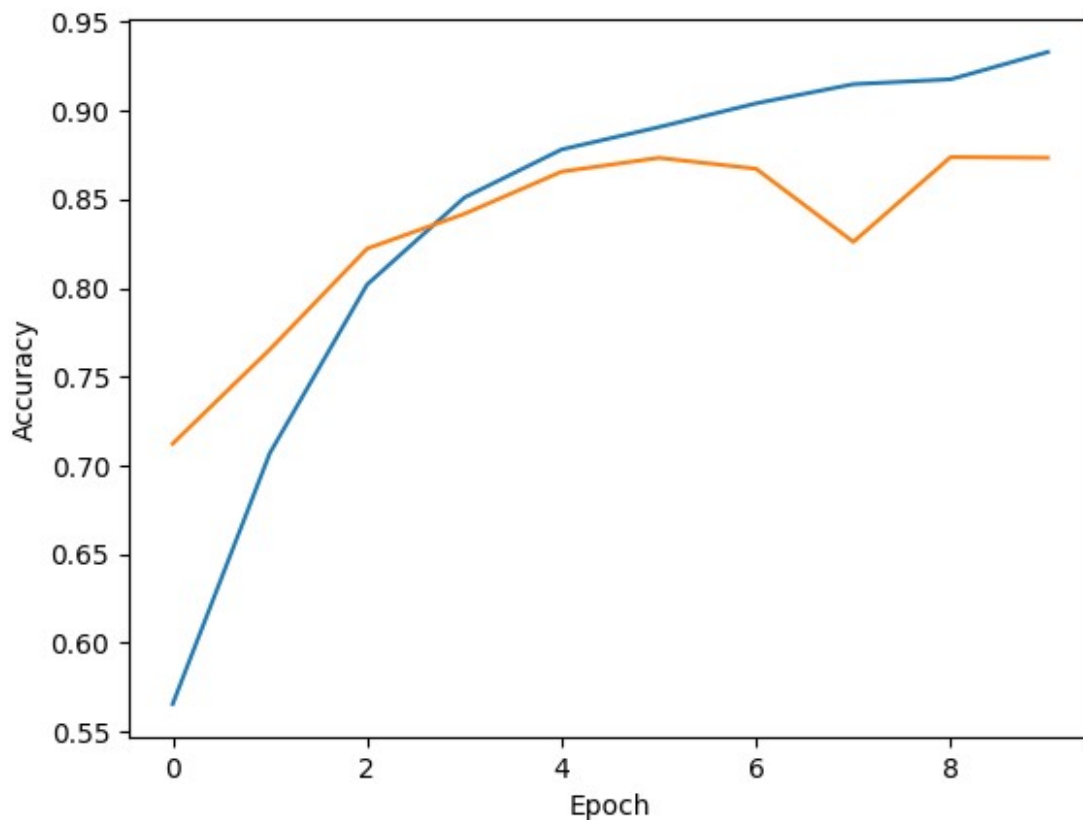
40/40 ————— 14s 349ms/step - accuracy: 0.9042 - loss: 0.2468 - val_accuracy: 0.8738 - val_loss: 0.3161

Epoch 10/10

40/40 ————— 21s 356ms/step - accuracy: 0.9362 - loss: 0.1853 - val_accuracy: 0.8734 - val_loss: 0.3257

```
import matplotlib.pyplot as plt

plt.plot(hist.history['accuracy'], label='accuracy')
plt.plot(hist.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
Text(0, 0.5, 'Accuracy')
```



```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# --- Evaluate model on test data ---
y_pred_probs = model_tf.predict(X_test)
y_pred = (y_pred_probs > 0.5).astype("int32").flatten()

# --- Classification report ---
print("\n Classification Report:")
print(classification_report(y_test, y_pred, target_names=["Negative",
```

```

"Positive"]))

# --- Confusion matrix ---
cm = confusion_matrix(y_test, y_pred)
labels = ["Negative", "Positive"]

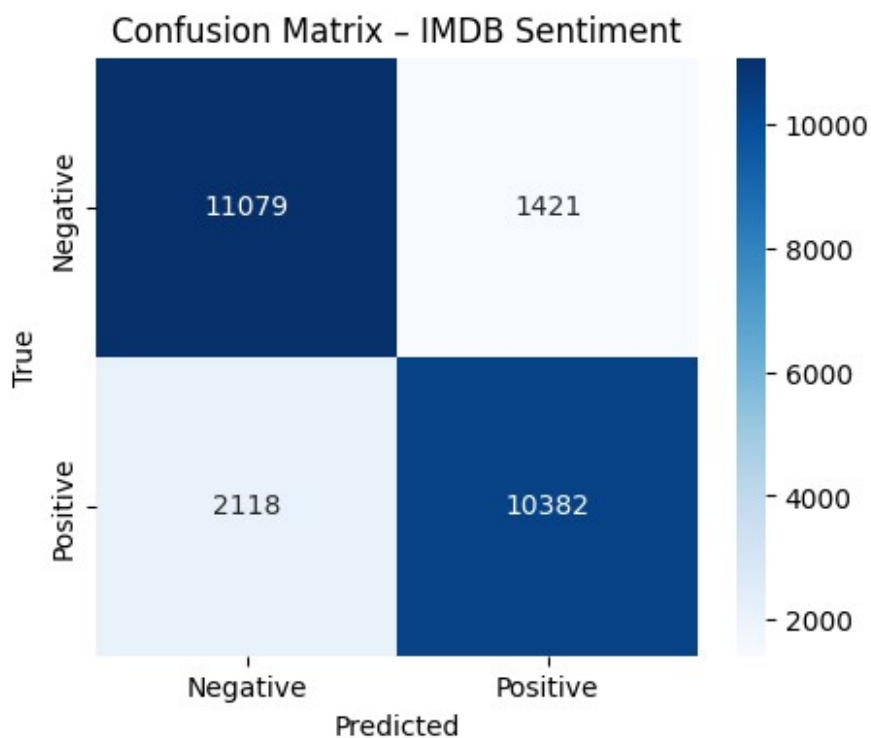
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix – IMDB Sentiment")
plt.show()

```

782/782 ————— 6s 8ms/step

□ Classification Report:

	precision	recall	f1-score	support
Negative	0.84	0.89	0.86	12500
Positive	0.88	0.83	0.85	12500
accuracy			0.86	25000
macro avg	0.86	0.86	0.86	25000
weighted avg	0.86	0.86	0.86	25000



Pytorch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# Convert to tensors
device = "cuda" if torch.cuda.is_available() else "cpu"
X_train_t = torch.tensor(X_train, dtype=torch.long)
y_train_t = torch.tensor(y_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.long)
y_test_t = torch.tensor(y_test, dtype=torch.float32)

train_loader = DataLoader(TensorDataset(X_train_t, y_train_t),
                           batch_size=512, shuffle=True)

# import torch
# import torch.nn as nn
# import torch.nn.functional as F

class SentimentTransformer(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, num_heads=4,
                  ff_dim=256, num_layers=1, max_len=256):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.pos_encoding = self._build_positional_encoding(max_len,
                                                             embed_dim)
        self.encoder_layer = nn.TransformerEncoderLayer(d_model=embed_dim,
                                                         nhead=num_heads,
                                                         dim_feedforward=ff_dim, batch_first=True)
        self.transformer = nn.TransformerEncoder(self.encoder_layer,
                                                  num_layers=num_layers)
        self.fc = nn.Linear(embed_dim, 1)

    def _build_positional_encoding(self, max_len, d_model):
        pos = torch.arange(0, max_len).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2) * (-
            torch.log(torch.tensor(10000.0)) / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        return pe.unsqueeze(0) # shape (1, max_len, d_model)

    def forward(self, x):
        seq_len = x.size(1)
        x = self.embedding(x) +
self.pos_encoding[:, :seq_len, :].to(x.device)
        x = self.transformer(x)
        x = x.mean(dim=1) # Global average pooling
```

```

        x = torch.sigmoid(self.fc(x))
        return x.squeeze(-1)

model_torch = SentimentTransformer(max_words).to(device)
optimizer = optim.Adam(model_torch.parameters(), lr=1e-3)
criterion = nn.BCELoss()

# --- Training loop ---
for epoch in range(10):
    model_torch.train()
    total_loss = 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        pred = model_torch(xb).squeeze()
        loss = criterion(pred, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}: loss={total_loss/len(train_loader):.4f}")

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import torch

# --- Evaluate on the test set ---
model_torch.eval()
y_true, y_pred = [], []

with torch.no_grad():
    for i in range(0, len(X_test_t), 512):
        xb = X_test_t[i:i+512].to(device)
        preds = model_torch(xb).squeeze().cpu()
        y_pred.extend((preds > 0.5).int().numpy())
        y_true.extend(y_test_t[i:i+512].numpy())

# # --- Classification report ---
# print("\n Classification Report:")
# print(classification_report(y_true, y_pred,
#                             target_names=["Negative", "Positive"]))

# # --- Confusion matrix ---
# cm = confusion_matrix(y_true, y_pred)
# labels = ["Negative", "Positive"]

# plt.figure(figsize=(5, 4))
# sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
#             xticklabels=labels, yticklabels=labels)

```

```

# plt.xlabel("Predicted")
# plt.ylabel("True")
# plt.title("Confusion Matrix – PyTorch IMDB Sentiment")
plt.show()

import torch
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_torch.eval()

# --- Create dataset & dataloader ---
test_ds = TensorDataset(X_test_t, y_test_t)
test_loader = DataLoader(test_ds, batch_size=512, shuffle=False)

# --- Evaluation ---
y_true, y_pred = [], []

with torch.no_grad():
    for xb, yb in test_loader:
        xb, yb = xb.to(device), yb.to(device)
        preds = model_torch(xb).squeeze()
        y_true.extend(yb.cpu().numpy())
        y_pred.extend((preds > 0.5).int().cpu().numpy())

# --- Metrics ---
print(classification_report(y_true, y_pred, target_names=["negative",
"positive"]))

# --- Confusion matrix ---
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["Pred Neg", "Pred Pos"],
            yticklabels=["True Neg", "True Pos"])
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - PyTorch IMDB Sentiment")
plt.show()

```

TODO

Efficiency of determined classifier depend on several parameters, related to:

- Preprocessing: sequence length and vocabulary size
- Architecture: embedding dimension, number of heads, size of the hidden layer (feed forward dimensionon)

- Training process: number of epochs and learning rate

Prepare 8 configurations (including various preprocessing parameters). Report results in the table summarizing parameter values and classification results (report accuracy, precision, recall, F1).

Display a few reviews from the dataset and print classification result

```
# Efficiency of determined classifier depend on several parameters, related to:

# Preprocessing: sequence length and vocabulary size
# Architecture: embedding dimension, number of heads, size of the hidden layer (feed forward dimensionon)
# Training process: number of epochs and learning rate
# Prepare 8 configurations (including various preprocessing parameters). Report results in the table summarizing parameter values and classification results (report accuracy, precision, recall, F1).

# Display a few reviews from the dataset and print classification result

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def run_model(seq_len, vocab_size, embed_dim, heads, ff_dim, epochs, lr):

    test_ds = TensorDataset(X_test_t, y_test_t)
    test_loader = DataLoader(test_ds, batch_size=512, shuffle=False)

    model = SentimentTransformer(
        vocab_size=vocab_size,
        embed_dim=embed_dim,
        num_heads=heads,
        ff_dim=ff_dim,
        num_layers=1,
        max_len=X_train.shape[1]
    ).to(device)

    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.BCELoss()

    for epoch in range(epochs):
        model.train()
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            xb = torch.clamp(xb, max=vocab_size-1)
```

```

        pred = model(xb).squeeze()
        loss = criterion(pred, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    model.eval()
    y_true, y_pred = [], []
    with torch.no_grad():
        for xb, yb in test_loader:
            xb, yb = xb.to(device), yb.to(device)
            xb = torch.clamp(xb, max=vocab_size-1)
            preds = model(xb).squeeze()
            y_true.extend(yb.cpu().numpy())
            y_pred.extend((preds > 0.5).int().cpu().numpy())

    random_idx = np.random.randint(0, len(X_test))
    review_tokens = X_test_t[random_idx].unsqueeze(0).to(device)
    true_val = y_test_t[random_idx].item()
    pred = model(review_tokens).item()
    print(f"Test text: {test_texts[random_idx]}")
    print("Prediction:", "Positive" if pred > 0.5 else "Negative ",
    f"True value: ", "Positive" if true_val == 1.0 else "Negative")
    print("Score:", round(pred, 4), "\n")

    return {
        "accuracy" : accuracy_score(y_true, y_pred),
        "precision" : precision_score(y_true, y_pred),
        "recall" : recall_score(y_true, y_pred),
        "f1" : f1_score(y_true, y_pred)
    }

%env CUDA_LAUNCH_BLOCKING=1

# I didn't change the sequence length, because I've got some Cuda
erros, that I couldn't fix...

configs = [
    # Conf_name, seq_len, vocab_size, embed_dim, heads, ff_dim,
epochs, lr
    ("C1", 128, 10000, 64, 2, 128, 5, 1e-3),
    ("C2", 128, 10000, 128, 4, 256, 5, 1e-3),
    ("C3", 256, 10000, 64, 4, 256, 10, 1e-3),
    ("C4", 256, 10000, 128, 4, 256, 5, 5e-4),
    ("C5", 128, 10000, 128, 2, 128, 10, 1e-3),
    ("C6", 256, 10000, 64, 4, 128, 10, 5e-4),
    ("C7", 256, 10000, 128, 4, 256, 10, 5e-4),
    ("C8", 128, 10000, 64, 2, 128, 5, 5e-4)

```

```

]

# For readability purposes I rewrite this configs, but I wont run this
# code once again, it will take too long...

# configs_dict = [
#     {"Conf_name": "C1", "seq_len": 128, "vocab_size": 10000,
#      "embed_dim": 64, "heads": 2, "ff_dim":128, "epochs": 5, "lr":1e-3},
#     {"Conf_name": "C2", "seq_len": 128, "vocab_size": 10000,
#      "embed_dim": 128, "heads": 4, "ff_dim":256, "epochs": 5, "lr":1e-3},
#     {"Conf_name": "C3", "seq_len": 256, "vocab_size": 10000,
#      "embed_dim": 64, "heads": 4, "ff_dim":256, "epochs": 10, "lr":1e-3},
#     {"Conf_name": "C4", "seq_len": 256, "vocab_size": 10000,
#      "embed_dim": 128, "heads": 4, "ff_dim":256, "epochs": 5, "lr":5e-4},
#     {"Conf_name": "C5", "seq_len": 128, "vocab_size": 10000,
#      "embed_dim": 128, "heads": 2, "ff_dim":128, "epochs": 10, "lr":1e-3},
#     {"Conf_name": "C6", "seq_len": 256, "vocab_size": 10000,
#      "embed_dim": 64, "heads": 4, "ff_dim":128, "epochs": 10, "lr":5e-4},
#     {"Conf_name": "C7", "seq_len": 256, "vocab_size": 10000,
#      "embed_dim": 128, "heads": 4, "ff_dim":256, "epochs": 10, "lr":5e-4},
#     {"Conf_name": "C8", "seq_len": 128, "vocab_size": 10000,
#      "embed_dim": 64, "heads": 2, "ff_dim":128, "epochs": 5, "lr":5e-4}
# ]

results = {}
for name, seq_len, vocab, emb, heads, ff, epochs, lr in configs:
    print(f"\nRunning {name}...")
    results[name] = run_model(seq_len, vocab, emb, heads, ff, epochs,
lr)

import pandas as pd
df_results = pd.DataFrame(results).T
df_results

sample_ids = [11, 123, 204, 321, 5]

for i in sample_ids:
    review_tokens = X_test_t[i].unsqueeze(0).to(device)
    true_val = y_test_t[i].item()
    pred = model_torch(review_tokens).item()

    print(f"Test text: {test_texts[i]}")
    print("Prediction:", "Positive" if pred > 0.5 else "Negative ",
f"True value: {true_val}")
    print("Score:", round(pred, 4), "\n")

```

3.2 Visual transformer (ViT)

Vision Transformers (ViTs) are models that apply the Transformer architecture directly to images by representing them as sequences of patch embeddings, allowing self-attention to capture relationships across the entire image without relying on convolutional operations.

Patches

Instead of tokens from text, ViT creates “visual tokens.” An input image of shape $H \times W \times C$ is split into fixed size non-overlapping patches of size $P \times P \times C$. Hence, an image is represented by a sequence of visual tokens of length N , where $N = \frac{H \times W}{P^2}$.

Next, each patch $x_i \in R^{P \times P \times C}$ is flattened into a vector:

$$x_i \rightarrow \text{vec}(x_i) \in R^{P^2 C}$$

Linear patch embedding

Each flattened patch is linearly projected to a fixed-dimensional embedding space of dimension D :

$$z_i = W_E x_i + b_E, W_E \in R^{D \times (P^2 C)}$$

This produces a sequence of embedded tokens:

$$Z = [z_1, z_2, \dots, z_N]$$

The matrix W_E is learnable.

Class Token

A learnable *classification token* $\mathbf{z}_{\text{CLS}} \in \mathbb{R}^D$ can be inserted at the beginning of the sequence:

$$Z' = [z_{\text{CLS}}, z_1, \dots, z_N]$$

Positional Embedding

Transformer has no inherent notion of spatial order. Information on positions of patches in the image is encoded as embedding vectors.

Patch positions are indexed in images along rows or columns, and form a sequences of size N , e.g:

(0,0), (0,1), (0,2), ..., (1,0), (1,1), ...

Typically, learnable embeddings $\mathbf{PE} \in \mathbb{R}^{(N+1) \times D}$ are used.

$$Z'' = Z' + PE$$

Transformer Encoder Block

- Transformer Encoder Block
 - Self-Attention Block
 - MLP (Multilayer Perceptron)

Transformer Encoder Blocks can be repeated several times. It is believed that subsequent blocks catch higher level features.

The output is a sequence of hidden states $H_{\text{enc}} = [h_{\text{CLS}}, h_1, \dots, h_n, h_{\text{SEP}}]$

Dense and output layers

If class token is used, its value h_{CLS} is extracted. In the opposite case mean value (sometimes all values) of $h_1, \dots, h_n$ is used in the final block making classification.

3.2.1 Example of patches (CIFAR 10)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

(x_train, y_train), (x_test, y_test) =
keras.datasets.cifar10.load_data()

print(f"Train: {x_train.shape}, Test: {x_test.shape}")
# (50000, 32, 32, 3) and (10000, 32, 32, 3)

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

import matplotlib.pyplot as plt

fig, axes = plt.subplots(8, 8, figsize=(6, 6))
for i, ax in enumerate(axes.flat):
    patch = x_train[i]
    ax.imshow(patch)
    ax.axis("off")

plt.tight_layout()
plt.show()

class Patches(layers.Layer):
```

```

def __init__(self, patch_size):
    super().__init__()
    self.patch_size = patch_size

def call(self, images):
    batch_size = tf.shape(images)[0]
    patches = tf.image.extract_patches(
        images=images,
        sizes=[1, self.patch_size, self.patch_size, 1],
        strides=[1, self.patch_size, self.patch_size, 1],
        rates=[1, 1, 1, 1],
        padding='VALID'
    )
    patch_dims = tf.shape(patches)[-1]
    patches = tf.reshape(patches, [batch_size, -1, patch_dims])
    return patches

xp = Patches(patch_size=patch_size)(x_train[19].reshape(1, 32, 32, 3))

import matplotlib.pyplot as plt
import tensorflow as tf

patch_size = 4

xp = Patches(patch_size=patch_size)(x_train[7].reshape(1, 32, 32, 3))

patches = xp[0].numpy().reshape(-1, patch_size, patch_size, 3)

fig, axes = plt.subplots(x_train.shape[1]//patch_size,
x_train.shape[1]//patch_size, figsize=(8, 8))
for i, ax in enumerate(axes.flat):
    patch = patches[i]
    ax.imshow(patch)
    ax.axis("off")

plt.tight_layout()
plt.show()

```

3.2.1 Fashion MNIST example (Tensorflow)

In this example class token is not used. Instead, all values of hidden states are used for final classification.

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# --- 1. Load Fashion-MNIST ---
(x_train, y_train), (x_test, y_test) =

```

```

keras.datasets.fashion_mnist.load_data()
x_train = x_train[..., None] / 255.0
x_test = x_test[..., None] / 255.0

# --- 2. Define patch extraction and encoder layers ---
patch_size = 4
num_patches = (x_train.shape[1] // patch_size) ** 2
projection_dim = 64

class Patches(layers.Layer):
    def __init__(self, patch_size):
        super().__init__()
        self.patch_size = patch_size

    def call(self, images):
        batch_size = tf.shape(images)[0]
        patches = tf.image.extract_patches(
            images=images,
            sizes=[1, self.patch_size, self.patch_size, 1],
            strides=[1, self.patch_size, self.patch_size, 1],
            rates=[1, 1, 1, 1],
            padding='VALID'
        )
        patch_dims = tf.shape(patches)[-1]
        patches = tf.reshape(patches, [batch_size, -1, patch_dims])
        return patches

class PatchEncoder(layers.Layer):
    def __init__(self, num_patches, projection_dim):
        super().__init__()
        self.num_patches = num_patches
        self.projection = layers.Dense(units=projection_dim)
        self.position_embedding =
layers.Embedding(input_dim=num_patches, output_dim=projection_dim)

    def call(self, patches):
        x = self.projection(patches)
        positions = tf.range(start=0, limit=self.num_patches, delta=1)
        pos_emb = self.position_embedding(positions)
        pos_emb = tf.expand_dims(pos_emb, axis=0)
        return x + pos_emb

# --- 3. Build ViT model ---
def create_vit_classifier():
    inputs = layers.Input(shape=(28,28,1))
    patches = Patches(patch_size)(inputs)
    encoded_patches = PatchEncoder(num_patches, projection_dim)
    (patches)

# Transformer blocks

```

```

for _ in range(4):
    # LayerNorm + Multi-Head Self Attention
    x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
    attention_output = layers.MultiHeadAttention(
        num_heads=4, key_dim=projection_dim, dropout=0.1
    )(x1, x1)
    x2 = layers.Add()([attention_output, encoded_patches])

    # MLP block
    x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
    x3 = layers.Dense(projection_dim * 2, activation=tf.nn.gelu)
(x3)
    x3 = layers.Dense(projection_dim)(x3)
    encoded_patches = layers.Add()([x3, x2])

    # Classification head
    representation = layers.LayerNormalization(epsilon=1e-6)
(encoded_patches)
    representation = layers.Flatten()(representation)
    representation = layers.Dropout(0.5)(representation)
    logits = layers.Dense(10)(representation)

    return keras.Model(inputs=inputs, outputs=logits)

model = create_vit_classifier()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)

# --- 4. Train ---
model.fit(x_train, y_train, batch_size=128, epochs=20,
validation_split=0.1)

# --- 5. Evaluate ---
model.evaluate(x_test, y_test)

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Class names for Fashion-MNIST ---
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

```

```

# --- 2. Predict on test set ---
y_pred_logits = model.predict(x_test)
y_pred = np.argmax(y_pred_logits, axis=1)

# --- 3. Classification report ---
print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=class_names))

# --- 4. Confusion matrix ---
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.title("Fashion-MNIST Confusion Matrix")
plt.show()

```

GELU is a smooth function similar to ReLU commonly used in Transformer models

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erf

# Exact GELU
def gelu(x):
    return 0.5 * x * (1 + erf(x / np.sqrt(2)))

x = np.linspace(-3, 3, 500)
y = gelu(x)

plt.plot(x, y, label="GELU")
plt.plot(x, np.maximum(0, x), '--', label="ReLU")
plt.title("GELU vs ReLU")
plt.xlabel("x")
plt.ylabel("activation")
plt.grid(True)
plt.legend()
plt.show()

```

3.2.2 Fashion MNIST with class token (TF)

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# --- 1. Load Fashion-MNIST ---
(x_train, y_train), (x_test, y_test) =
keras.datasets.fashion_mnist.load_data()

```

```

x_train = x_train[..., None] / 255.0
x_test = x_test[..., None] / 255.0

# --- 2. Define patch extraction and encoder layers ---
patch_size = 4
num_patches = (x_train.shape[1] // patch_size) ** 2
projection_dim = 64

class Patches(layers.Layer):
    def __init__(self, patch_size):
        super().__init__()
        self.patch_size = patch_size

    def call(self, images):
        batch_size = tf.shape(images)[0]
        patches = tf.image.extract_patches(
            images=images,
            sizes=[1, self.patch_size, self.patch_size, 1],
            strides=[1, self.patch_size, self.patch_size, 1],
            rates=[1, 1, 1, 1],
            padding='VALID'
        )
        patch_dims = tf.shape(patches)[-1]
        patches = tf.reshape(patches, [batch_size, -1, patch_dims])
        return patches

class PatchEncoder(layers.Layer):
    def __init__(self, num_patches, projection_dim):
        super().__init__()
        self.num_patches = num_patches
        self.projection = layers.Dense(units=projection_dim)
        self.position_embedding = layers.Embedding(
            input_dim=num_patches + 1, # +1 na class token
            output_dim=projection_dim
        )
        # Inicjalizujemy class token jako trenowalny wektor
        self.class_token = self.add_weight(
            name="class_token",
            shape=(1, 1, projection_dim),
            initializer="random_normal",
            trainable=True,
        )

    def call(self, patches):
        batch_size = tf.shape(patches)[0]
        x = self.projection(patches)

        # Broadcast class token for each batch element
        class_token = tf.broadcast_to(self.class_token, [batch_size,

```

```

1, projection_dim])

    # Class token is inserted at the beginning
    x = tf.concat([class_token, x], axis=1)

    # Token positions (add 1 because of the class token )
    positions = tf.range(start=0, limit=self.num_patches + 1,
delta=1)
    pos_emb = self.position_embedding(positions)
    pos_emb = tf.expand_dims(pos_emb, axis=0)
    return x + pos_emb

# --- 3. Build ViT model ---
def create_vit_classifier():
    inputs = layers.Input(shape=(28, 28, 1))
    patches = Patches(patch_size)(inputs)
    encoded_patches = PatchEncoder(num_patches, projection_dim)
    (patches)

    # Transformer blocks
    for _ in range(4):
        x1 = layers.LayerNormalization(epsilon=1e-6)(encoded_patches)
        attention_output = layers.MultiHeadAttention(
            num_heads=4, key_dim=projection_dim, dropout=0.1
        )(x1, x1)
        x2 = layers.Add()([attention_output, encoded_patches])

        x3 = layers.LayerNormalization(epsilon=1e-6)(x2)
        x3 = layers.Dense(projection_dim * 2, activation=tf.nn.gelu)
    (x3)
        x3 = layers.Dense(projection_dim)(x3)
        encoded_patches = layers.Add()([x3, x2])

    # Classification head (tylko class token)
    representation = layers.LayerNormalization(epsilon=1e-6)
    (encoded_patches)
    # extract the first token = class token
    class_token_output = representation[:, 0, :]
    representation = layers.Dropout(0.5)(class_token_output)
    logits = layers.Dense(10)(representation)

    return keras.Model(inputs=inputs, outputs=logits)

# --- 4. Compile & train ---
model = create_vit_classifier()
model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),

```

```

        metrics=["accuracy"],
    )

model.summary()

# --- 5. Train ---
model.fit(x_train, y_train, batch_size=128, epochs=20,
validation_split=0.1)

# --- 6. Evaluate ---
model.evaluate(x_test, y_test)

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Class names for Fashion-MNIST ---
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

# --- 2. Predict on test set ---
y_pred_logits = model.predict(x_test)
y_pred = np.argmax(y_pred_logits, axis=1)

# --- 3. Classification report ---
print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=class_names))

# --- 4. Confusion matrix ---
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.title("Fashion-MNIST Confusion Matrix")
plt.show()

```

3.2.3 Fashion MNIST Pytorch (with class token)

```

import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

```

```

# --- Data ---
transform = transforms.Compose([
    # transforms.Resize(32),
    transforms.ToTensor()
])
train_data = datasets.FashionMNIST(root='.', train=True,
download=True, transform=transform)
test_data = datasets.FashionMNIST(root='.', train=False,
transform=transform)
train_loader = DataLoader(train_data, batch_size=128, shuffle=True)
test_loader = DataLoader(test_data, batch_size=128)

# --- Model ---
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels=1, patch_size=4, emb_size=128,
img_size=28, use_cls_token=True):
        super().__init__()
        self.proj = nn.Conv2d(in_channels, emb_size,
kernel_size=patch_size, stride=patch_size)
        num_patches = (img_size // patch_size) ** 2
        if use_cls_token:
            self.cls_token = nn.Parameter(torch.randn(1, 1, emb_size))
            self.pos_embed = nn.Parameter(torch.randn(1, num_patches +
1, emb_size))
        else:
            self.pos_embed = nn.Parameter(torch.randn(1, num_patches,
emb_size))

    def forward(self, x):
        B = x.shape[0]
        x = self.proj(x).flatten(2).transpose(1, 2) # (B,
num_patches, emb_size)
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)
        x = x + self.pos_embed
        return x

class ViT(nn.Module):
    def __init__(self, num_classes=10, emb_size=128, depth=4,
n_heads=8, patch_size=4, img_size=28, use_cls_token=True):
        super().__init__()
        self.patch_embed = PatchEmbedding(1, patch_size, emb_size,
img_size, use_cls_token)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=emb_size, nhead=n_heads,
dim_feedforward=emb_size*2, dropout=0.1, activation='gelu',
batch_first=True
        )
        self.encoder = nn.TransformerEncoder(encoder_layer,
num_layers=depth)

```

```

        self.head = nn.Linear(emb_size, num_classes)

    def forward(self, x):
        x = self.patch_embed(x)
        x = self.encoder(x)
        cls_output = x[:, 0]
        return self.head(cls_output)

# --- Training ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = ViT().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4,
weight_decay=1e-4)
criterion = nn.CrossEntropyLoss()

for epoch in range(20):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss:
{total_loss/len(train_loader):.4f}")

# --- Evaluation ---
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images).argmax(1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
print(f"Test Accuracy: {correct/total:.4f}")

import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import torch

# --- 1. Define class names ---
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"
]

```

```

# --- 2. Get predictions on test set ---
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        preds = torch.argmax(outputs, dim=1)
        all_preds.append(preds.cpu())
        all_labels.append(labels.cpu())

y_pred = torch.cat(all_preds).numpy()
y_true = torch.cat(all_labels).numpy()

# --- 3. Classification report ---
print("Classification Report:\n")
print(classification_report(y_true, y_pred, target_names=class_names))

# --- 4. Confusion matrix ---
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.title("Fashion-MNIST Confusion Matrix (PyTorch ViT)")
plt.show()

```

3.2.4 TODO

- Perform experiments on CIFAR-10 dataset. Observe, that image shapes are (32,32,3)
- Test at least 4 configurations using TF or Pytorch. What can be changed:
 - numbers of transformers blocks
 - embedding dimensions
 - using class tokens (or not)
 - patch size (2,4,8)
- Summarize results in a table.
- Include confusion matrix (use correct labels) and discuss which images were misclassified (e.g. aircrafts taken as frogs?)

```

class PatchEmbedding(nn.Module):
    def __init__(self, in_channels=1, patch_size=4, emb_size=128,
                 img_size=28, use_cls_token=True):
        super().__init__()
        self.proj = nn.Conv2d(in_channels, emb_size,
                              kernel_size=patch_size, stride=patch_size)

```

```

        num_patches = (img_size // patch_size) ** 2
        self.use_cls_token = use_cls_token
        if use_cls_token:
            self.cls_token = nn.Parameter(torch.randn(1, 1, emb_size))
            self.pos_embed = nn.Parameter(torch.randn(1, num_patches +
1, emb_size))
        else:
            self.pos_embed = nn.Parameter(torch.randn(1, num_patches,
emb_size))

    def forward(self, x):
        B = x.shape[0]
        x = self.proj(x).flatten(2).transpose(1, 2) # (B,
num_patches, emb_size)
        if self.use_cls_token:
            cls_tokens = self.cls_token.expand(B, -1, -1)
            x = torch.cat((cls_tokens, x), dim=1)
            x = x + self.pos_embed
        return x

class ViT(nn.Module):
    def __init__(self, num_classes=10, emb_size=128, depth=4, n_heads=8,
patch_size=4, img_size=28, use_cls_token=True):
        super().__init__()
        self.patch_embed = PatchEmbedding(1, patch_size, emb_size,
img_size, use_cls_token)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=emb_size, nhead=n_heads, dim_feedforward=emb_size*2,
dropout=0.1, activation='gelu', batch_first=True
        )
        self.encoder = nn.TransformerEncoder(encoder_layer,
num_layers=depth)
        self.head = nn.Linear(emb_size, num_classes)

    def forward(self, x):
        x = self.patch_embed(x)
        x = self.encoder(x)
        if self.patch_embed.use_cls_token:
            cls_output = x[:, 0]
        else:
            cls_output = x.mean(dim=1)
        return self.head(cls_output)

configs_vit = [

{"name": "C1", "depth": 2, "emb_size": 64, "patch_size": 4, "use_cls_token": Tr
ue},

{"name": "C2", "depth": 4, "emb_size": 128, "patch_size": 4, "use_cls_token": T
rue},

```

```

{"name": "C3", "depth": 6, "emb_size": 128, "patch_size": 2, "use_cls_token": False},

{"name": "C4", "depth": 8, "emb_size": 256, "patch_size": 8, "use_cls_token": True},
]

results = []

for cfg in configs_vit:
    print(f"Training config {cfg['name']}...")
    model = ViT(
        num_classes=10,
        emb_size=cfg['emb_size'],
        depth=cfg['depth'],
        patch_size=cfg['patch_size'],
        use_cls_token=cfg['use_cls_token']
    ).to(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4,
weight_decay=1e-4)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(5):
        model.train()
        total_loss = 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            loss = criterion(model(images), labels)
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        print(f"Epoch {epoch+1}, Loss:
{total_loss/len(train_loader):.4f}")

    # --- Evaluation ---
    model.eval()
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            preds = torch.argmax(outputs, dim=1)
            all_preds.append(preds.cpu())
            all_labels.append(labels.cpu())

```

```

y_pred = torch.cat(all_preds).numpy()
y_true = torch.cat(all_labels).numpy()

report_dict = classification_report(y_true, y_pred,
target_names=class_names, output_dict=True)
print("Classification Report:\n")
print(classification_report(y_true, y_pred,
target_names=class_names))

config_metrics = {
    'Config' : cfg['name'],
    'Accuracy': (y_pred == y_true).mean()
}

for cls in class_names:
    config_metrics[f'Precision_{cls}'] = report_dict[cls]['precision']
    config_metrics[f'Recall_{cls}'] = report_dict[cls]['recall']
    config_metrics[f'F1_{cls}'] = report_dict[cls]['f1-score']
    results.append(config_metrics)

acc = (y_pred == y_true).mean()
results.append({'Config':cfg["name"], "Accuracy":acc})

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=class_names, yticklabels=class_names)
plt.title(f"Confusion Matrix - Config {cfg['name']}")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.show()

df_results = pd.DataFrame(results)
df_results = df_results.drop_duplicates(subset='Config', keep='first')
df_results = df_results.reset_index(drop=True)
pd.set_option('display.max_columns', None)
# display(df_results)
df_results

```

3.3. Time series

In this section we will use Transformers for multivariate time series forecasting. The synthetic time series from Assignment 2 will be used

```

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

```

```

# --- parameters ---
n_agents = 1000
n_locations = 5
center = 2
steps_per_day = 24
n_days = 30
T = steps_per_day * n_days

# --- parameters controlling moves---
base_move_prob = 0.05
bias_to_center = 0.25
bias_to_home = 0.25
night_suppression = 0.2
random_variation = 0.05

# --- initiation of agents ---
home = np.random.choice(n_locations, n_agents)
location = home.copy()
x = np.zeros((T, n_locations))

def hour_phase(hour):
    if 7 <= hour < 9:
        return 'morning'
    elif 9 <= hour < 17:
        return 'day'
    elif 17 <= hour < 19:
        return 'evening'
    else:
        return 'night'

# --- main sim loop ---
for t in range(T):
    hour = t % steps_per_day
    phase = hour_phase(hour)

    # basic transition matrix
    P = np.ones((n_locations, n_locations)) * base_move_prob
    np.fill_diagonal(P, 1 - base_move_prob * (n_locations - 1))

    if phase == 'morning':
        P[:, center] += bias_to_center
    elif phase == 'evening':
        for i in range(n_agents):
            P[location[i], home[i]] += bias_to_home
    elif phase == 'night':
        P = 1 - night_suppression + night_suppression * P

    # random perturbations
    P += random_variation * (np.random.rand(*P.shape) - 0.5)

```

```

P = np.clip(P, 0, None)
P /= P.sum(axis=1, keepdims=True)

# updating agents locations
new_locations = []
for i in range(n_agents):
    loc = location[i]
    probs = P[loc]
    new_loc = np.random.choice(n_locations, p=probs)
    new_locations.append(new_loc)
location = np.array(new_locations)

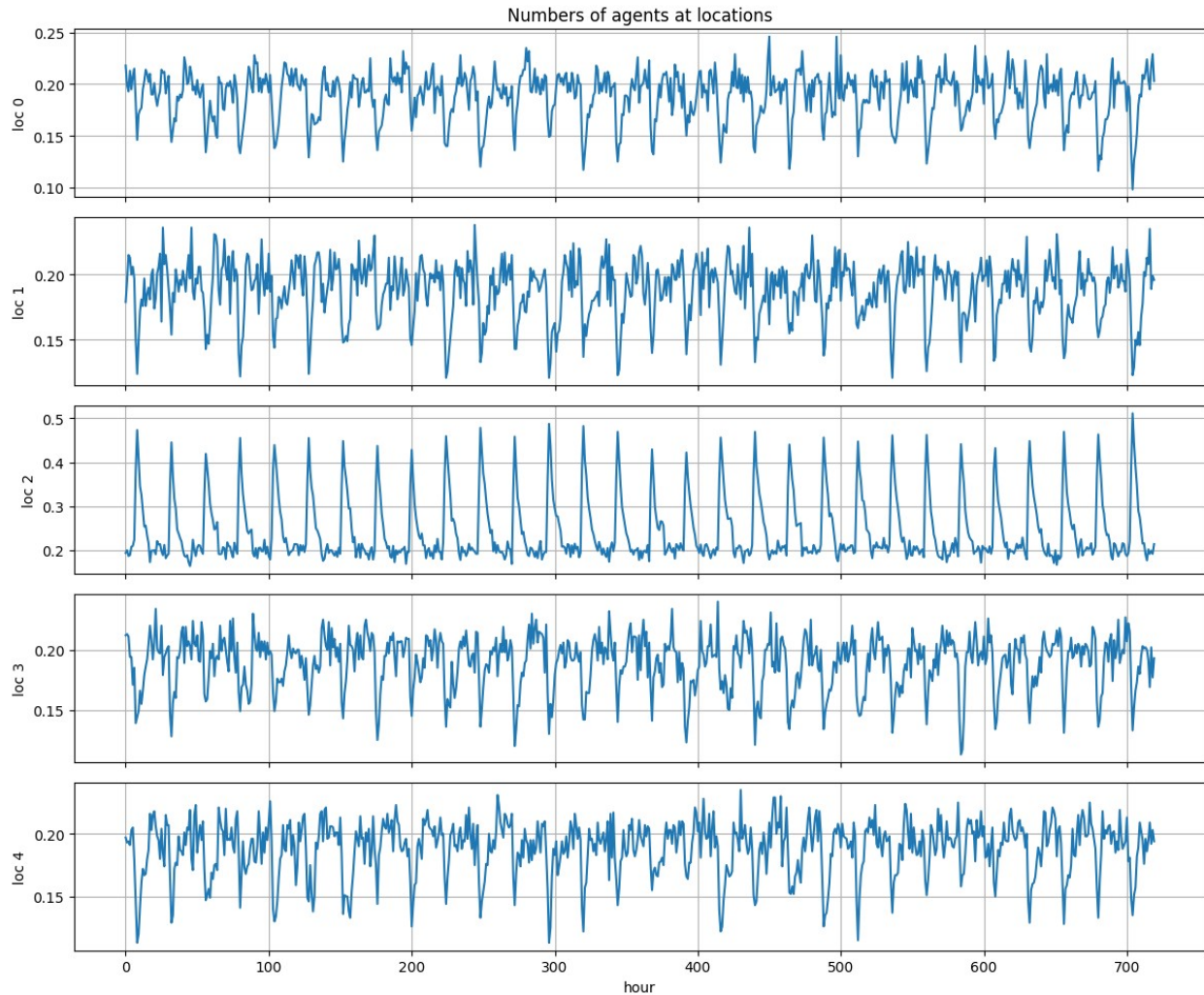
# saving states (with normalization)
for j in range(n_locations):
    x[t, j] = np.sum(location == j) / n_agents

# --- plots ---
fig, axes = plt.subplots(n_locations, 1, figsize=(12, 2*n_locations),
sharex=True)

for j in range(n_locations):
    axes[j].plot(x[:, j], color='tab:blue')
    axes[j].set_ylabel(f"loc {j}")
    axes[j].grid(True)
    if j == 0:
        axes[j].set_title("Numbers of agents at locations")

axes[-1].set_xlabel("hour")
plt.tight_layout()
plt.show()

```



```
import pandas as pd

# assume, x has the shape (T, n_locations)
df = pd.DataFrame(x, columns=[f"loc_{j}" for j in range(n_locations)])
df.to_csv("population_timeseries.csv", index_label="t")
df.head()

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 720,\n  \"fields\": [\n    {\n      \"column\": \"loc_0\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02354707023580318,\n        \"min\": 0.098,\n        \"max\": 0.246,\n        \"num_unique_values\": 114,\n        \"samples\": [\n          0.18,\n          0.195,\n          0.168\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"loc_1\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.02241805658243809,\n        \"min\": 0.121,\n        \"max\": 0.238,\n        \"num_unique_values\": 109,\n        \"samples\": [\n          0.238,\n          0.172,\n          0.2\n        ]\n      }\n    }\n  ]\n}
```

```

],\n      \"semantic_type\": \"\", \n      \"description\": \"\"\n}\n  },\n  {\n    \"column\": \"loc_2\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\":\n0.07401748087369608, \n      \"min\": 0.164, \n      \"max\":\n0.511, \n      \"num_unique_values\": 219, \n      \"samples\": [\n0.299, \n      0.248, \n      0.442\n    ], \n    \"semantic_type\": \"\", \n    \"description\": \"\"\n  },\n  {\n    \"column\": \"loc_3\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\":\n0.022425470193815165, \n      \"min\": 0.113, \n      \"max\":\n0.24, \n      \"num_unique_values\": 107, \n      \"samples\": [\n0.217, \n      0.155, \n      0.171\n    ], \n    \"semantic_type\": \"\", \n    \"description\": \"\"\n  },\n  {\n    \"column\": \"loc_4\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\":\n0.022006568495482815, \n      \"min\": 0.113, \n      \"max\":\n0.235, \n      \"num_unique_values\": 107, \n      \"samples\": [\n0.126, \n      0.142, \n      0.203\n    ], \n    \"semantic_type\": \"\", \n    \"description\": \"\"\n  }\n]\n}","type":"dataframe","variable_name":"df"}

```

As previous we will use 24 hours window (past values) to predict values one step ahead. A dataset containing observations (x_i, y_i) where $x_i \in R^{5 \times 24}$ and $y_i \in R^5$ will be extracted from the time series.

```

import numpy as np

# parameters
window_size = 24 # use last 24 hours to predict one hour ahead
n_features = n_locations

# tworzymy X i y
X = []
y = []

for t in range(T - window_size):
    X.append(df.iloc[t:t+window_size].values)
    y.append(df.iloc[t+window_size].values)

X = np.array(X) # shape: (samples, window_size, n_features)
y = np.array(y) # shape: (samples, n_features)

print("X shape:", X.shape)
print("y shape:", y.shape)

X shape: (696, 24, 5)
y shape: (696, 5)

from sklearn.model_selection import train_test_split

```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)
```

Key point the time series data must be scaled.

```
from sklearn.preprocessing import StandardScaler

scaler_X = StandardScaler()
scaler_y = StandardScaler()

# Flatten time dimension for fitting (if your X is [samples, window,
# features])
X_train_2d = X_train.reshape(-1, X_train.shape[-1])
scaler_X.fit(X_train_2d)

# Scale y as well
scaler_y.fit(y_train)

StandardScaler()

X_train_scaled = scaler_X.transform(X_train_2d).reshape(X_train.shape)
X_test_scaled = scaler_X.transform(X_test.reshape(-1, X_test.shape[-1])).reshape(X_test.shape)

y_train_scaled = scaler_y.transform(y_train)
y_test_scaled = scaler_y.transform(y_test)
```

3.3.1 TensorFlow implementation

Key points

- Use sin/cos positional encoding
- Do not use class token. The last token in the sequence is used in prediction (or mean of token values - commented out `GlobalAveragePooling1D` or all token values)

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras import metrics

# === Hyperparameters ===
window_size = 24
num_features = 5
d_model = 64
num_heads = 4
ff_dim = 128
num_layers = 2

# === Positional Encoding ===
```

```

class PositionalEncoding(layers.Layer):
    def __init__(self, maxlen, d_model):
        super().__init__()
        self.maxlen = maxlen
        self.d_model = d_model
        # prepare positional encoding only once (float32)
        pos = tf.cast(tf.range(self.maxlen)[:], tf.newaxis],
tf.float32) # (maxlen, 1)
        i = tf.cast(tf.range(self.d_model)[tf.newaxis, :], tf.float32)
# (1, d_model)
        # frequencies: 1 / (10000^(2*(i//2)/d_model))
        angle_rates = 1.0 / tf.pow(10000.0, (2.0 * tf.floor(i / 2.0))
/ tf.cast(self.d_model, tf.float32))
        angle_rads = pos * angle_rates # (maxlen, d_model)

        # sin for even indexes, cos for odd
        sines = tf.sin(angle_rads[:, 0::2]) # (maxlen, d_model/2)
        cosines = tf.cos(angle_rads[:, 1::2]) # (maxlen, d_model/2)

        # arange as [sin0, cos0, sin1, cos1, ...]
        pos_encoding = tf.reshape(tf.stack([sines, cosines], axis=-1),
(self.maxlen, self.d_model))
        pos_encoding = pos_encoding[tf.newaxis, ...] # (1, maxlen,
d_model)
        self.pos_encoding = tf.cast(pos_encoding, tf.float32)

    def call(self, x):
        # x: (batch, seq_len, d_model_in)
        seq_len = tf.shape(x)[1]
        # cast position dtype p dtype of input and add
        return x + tf.cast(self.pos_encoding[:, :seq_len, :], x.dtype)

# === Transformer Block ===
def transformer_block(inputs):
    attn_output = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=d_model)(inputs, inputs)
    attn_output = layers.Dropout(0.1)(attn_output)
    out1 = layers.LayerNormalization(epsilon=1e-6)(inputs +
attn_output)

    ffn = tf.keras.Sequential([
        layers.Dense(ff_dim, activation='relu'),
        layers.Dense(d_model),
    ])
    ffn_output = ffn(out1)
    ffn_output = layers.Dropout(0.1)(ffn_output)
    return layers.LayerNormalization(epsilon=1e-6)(out1 + ffn_output)

# === Full Model ===

```

```

inputs = layers.Input(shape=(window_size, num_features))

# Project features to model dimension
x = layers.Dense(d_model)(inputs)
x = PositionalEncoding(window_size, d_model)(x)

# Apply N transformer layers
for _ in range(num_layers):
    x = transformer_block(x)

# Use the last token
last = layers.Lambda(lambda t: t[:, -1, :])(x)
outputs = layers.Dense(num_features)(last)

# Or pool across the time dimension
# x = layers.GlobalAveragePooling1D()(x)
# outputs = layers.Dense(num_features)(x)

# Or use the whole sequence
# x = layers.Flatten()(x)
# outputs = layers.Dense(num_features)(x)

model = models.Model(inputs, outputs)
model.compile(optimizer='adam', loss='mse', metrics=[
    metrics.MeanAbsoluteError(name='mae'),
    metrics.RootMeanSquaredError(name='rmse'),
    metrics.R2Score(name='r2')])

model.summary()

hist = model.fit(
    X_train_scaled, y_train_scaled,
    validation_data=(X_test_scaled, y_test_scaled),
    epochs=50,
    batch_size=32
)

loss, mae, rmse, r2 = model.evaluate(X_test_scaled, y_test_scaled)
print(f"Test MAE: {mae} Test RMSE: {rmse} Test r2 {r2}")

plt.plot(hist.history['loss'], label='loss')
plt.plot(hist.history['val_loss'], label='val_loss')
plt.legend()
plt.show()

hist.history.keys()

plt.plot(hist.history['r2'], label='r2')
plt.plot(hist.history['val_r2'], label='val_r2')

```

```

plt.legend()
plt.show()

y_pred_scaled = model.predict(X_test_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled)

from sklearn.metrics import r2_score, mean_squared_error
print("R²:", r2_score(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))

import matplotlib.pyplot as plt
import numpy as np

# assume you already have:
# y_test: shape (N, 5)
# y_pred: shape (N, 5)

n_features = y_test.shape[1]
timesteps = np.arange(len(y_test))

fig, axes = plt.subplots(n_features, 1, figsize=(12, 2.5 *
n_features), sharex=True)

for i in range(n_features):
    ax = axes[i]
    ax.plot(timesteps, y_test[:, i], label='True', linewidth=1.5)
    ax.plot(timesteps, y_pred[:, i], label='Pred', linestyle='--',
linewidth=1.5)
    ax.set_ylabel(f'loc_{i}')
    ax.grid(True, linestyle=':', alpha=0.7)
    if i == 0:
        ax.legend(loc='upper right')
    if i == n_features - 1:
        ax.set_xlabel('Time step')

plt.tight_layout()
plt.show()

```

3.3.2 PyTorch implementation

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader

# --- Configurable hyperparameters controlling model complexity ---
d_model = 64          # embedding dimension
num_heads = 4         # number of attention heads
num_layers = 4        # number of transformer encoder layers
ff_dim = 128          # feed-forward hidden dimension

```

```

dropout = 0.1          # dropout rate
batch_size = 32
epochs = 50
lr = 1e-3

# --- Prepare data ---
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

X_train = torch.tensor(X_train_scaled, dtype=torch.float32)
y_train = torch.tensor(y_train_scaled, dtype=torch.float32)
X_val = torch.tensor(X_test_scaled, dtype=torch.float32)
y_val = torch.tensor(y_test_scaled, dtype=torch.float32)

train_loader = DataLoader(TensorDataset(X_train, y_train),
                           batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val),
                        batch_size=batch_size, shuffle=False)

class TimeSeriesTransformer(nn.Module):
    def __init__(self, input_dim, d_model, n_heads, n_layers, ff_dim,
                  dropout=0.1):
        super().__init__()

        # Project input features (e.g. 5 → 64)
        self.input_proj = nn.Linear(input_dim, d_model)

        # Positional encoding
        self.pos_encoding = self._generate_pos_encoding(500, d_model)

        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=ff_dim,
            dropout=dropout,
            batch_first=True,
            activation='gelu'
        )
        self.transformer = nn.TransformerEncoder(encoder_layer,
                                                  num_layers=n_layers)

        # Regression head
        self.head = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, 64),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(64, input_dim)
        )

```

```

def _generate_pos_encoding(self, max_len, d_model):
    """Precompute sinusoidal positional encodings"""
    position = torch.arange(max_len).unsqueeze(1)
    div_term = torch.exp(torch.arange(0, d_model, 2) * (-
torch.log(torch.tensor(10000.0)) / d_model))
    pe = torch.zeros(max_len, d_model)
    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)
    pe = pe.unsqueeze(0)
    return pe

def forward(self, x):
    seq_len = x.size(1)
    x = self.input_proj(x) +
self.pos_encoding[:, :seq_len, :].to(x.device)
    x = self.transformer(x)
    x = x.mean(dim=1) # Global average pooling over sequence
    return self.head(x)

model = TimeSeriesTransformer(
    input_dim=X_train.shape[2],
    d_model=d_model,
    n_heads=num_heads,
    n_layers=num_layers,
    ff_dim=ff_dim,
    dropout=dropout
).to(device)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

def r2_score(y_true, y_pred):
    ss_res = torch.sum((y_true - y_pred) ** 2)
    ss_tot = torch.sum((y_true - torch.mean(y_true)) ** 2)
    return 1 - ss_res / ss_tot

for epoch in range(epochs):
    model.train()
    train_loss = 0
    for Xb, yb in train_loader:
        Xb, yb = Xb.to(device), yb.to(device)
        optimizer.zero_grad()
        preds = model(Xb)
        loss = criterion(preds, yb)
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * Xb.size(0)

    model.eval()
    val_loss, val_r2 = 0, 0

```

```

with torch.no_grad():
    for Xb, yb in val_loader:
        Xb, yb = Xb.to(device), yb.to(device)
        preds = model(Xb).squeeze()
        val_loss += criterion(preds, yb).item() * Xb.size(0)
        val_r2 += r2_score(yb, preds).item() * Xb.size(0)

train_loss /= len(train_loader.dataset)
val_loss /= len(val_loader.dataset)
val_r2 /= len(val_loader.dataset)

print(f"Epoch {epoch+1:02d} | Train loss: {train_loss:.4f} | "
      f"Val loss: {val_loss:.4f} | Val R2: {val_r2:.4f}")

```

Epoch 01	Train loss: 0.9967	Val loss: 0.8527	Val R2: 0.1744
Epoch 02	Train loss: 0.6337	Val loss: 0.4813	Val R2: 0.5190
Epoch 03	Train loss: 0.4244	Val loss: 0.3851	Val R2: 0.6045
Epoch 04	Train loss: 0.3787	Val loss: 0.3336	Val R2: 0.6522
Epoch 05	Train loss: 0.3687	Val loss: 0.3396	Val R2: 0.6486
Epoch 06	Train loss: 0.3457	Val loss: 0.3340	Val R2: 0.6589
Epoch 07	Train loss: 0.3382	Val loss: 0.3107	Val R2: 0.6714
Epoch 08	Train loss: 0.3375	Val loss: 0.3087	Val R2: 0.6768
Epoch 09	Train loss: 0.3256	Val loss: 0.3246	Val R2: 0.6653
Epoch 10	Train loss: 0.3173	Val loss: 0.3055	Val R2: 0.6787
Epoch 11	Train loss: 0.3110	Val loss: 0.3383	Val R2: 0.6528
Epoch 12	Train loss: 0.3128	Val loss: 0.3067	Val R2: 0.6796
Epoch 13	Train loss: 0.2956	Val loss: 0.3085	Val R2: 0.6754
Epoch 14	Train loss: 0.2978	Val loss: 0.3145	Val R2: 0.6739
Epoch 15	Train loss: 0.2939	Val loss: 0.3113	Val R2: 0.6744
Epoch 16	Train loss: 0.2942	Val loss: 0.3204	Val R2: 0.6714
Epoch 17	Train loss: 0.2802	Val loss: 0.2933	Val R2: 0.6914
Epoch 18	Train loss: 0.2699	Val loss: 0.3007	Val R2: 0.6853
Epoch 19	Train loss: 0.2704	Val loss: 0.2790	Val R2: 0.7038
Epoch 20	Train loss: 0.2757	Val loss: 0.2785	Val R2: 0.7062
Epoch 21	Train loss: 0.2615	Val loss: 0.2804	Val R2: 0.7060
Epoch 22	Train loss: 0.2501	Val loss: 0.2874	Val R2: 0.7019
Epoch 23	Train loss: 0.2487	Val loss: 0.2887	Val R2: 0.7006
Epoch 24	Train loss: 0.2457	Val loss: 0.2982	Val R2: 0.6944
Epoch 25	Train loss: 0.2511	Val loss: 0.2926	Val R2: 0.7019
Epoch 26	Train loss: 0.2480	Val loss: 0.2755	Val R2: 0.7169
Epoch 27	Train loss: 0.2387	Val loss: 0.2753	Val R2: 0.7181
Epoch 28	Train loss: 0.2269	Val loss: 0.2760	Val R2: 0.7218
Epoch 29	Train loss: 0.2345	Val loss: 0.2675	Val R2: 0.7262
Epoch 30	Train loss: 0.2233	Val loss: 0.3005	Val R2: 0.6993
Epoch 31	Train loss: 0.2195	Val loss: 0.2765	Val R2: 0.7098
Epoch 32	Train loss: 0.2179	Val loss: 0.2982	Val R2: 0.6998
Epoch 33	Train loss: 0.2137	Val loss: 0.2845	Val R2: 0.7128
Epoch 34	Train loss: 0.2164	Val loss: 0.2782	Val R2: 0.7156
Epoch 35	Train loss: 0.2212	Val loss: 0.2665	Val R2: 0.7271
Epoch 36	Train loss: 0.2114	Val loss: 0.2820	Val R2: 0.7083

Epoch 37	Train loss: 0.2065	Val loss: 0.2945	Val R2: 0.7017
Epoch 38	Train loss: 0.2073	Val loss: 0.2935	Val R2: 0.6990
Epoch 39	Train loss: 0.2077	Val loss: 0.3032	Val R2: 0.6922
Epoch 40	Train loss: 0.1994	Val loss: 0.3046	Val R2: 0.6943
Epoch 41	Train loss: 0.1968	Val loss: 0.2974	Val R2: 0.6986
Epoch 42	Train loss: 0.1946	Val loss: 0.2939	Val R2: 0.6946
Epoch 43	Train loss: 0.1930	Val loss: 0.3009	Val R2: 0.6974
Epoch 44	Train loss: 0.1858	Val loss: 0.3228	Val R2: 0.6723
Epoch 45	Train loss: 0.1883	Val loss: 0.3323	Val R2: 0.6618
Epoch 46	Train loss: 0.2049	Val loss: 0.3018	Val R2: 0.6890
Epoch 47	Train loss: 0.1881	Val loss: 0.2973	Val R2: 0.6960
Epoch 48	Train loss: 0.1846	Val loss: 0.3046	Val R2: 0.6840
Epoch 49	Train loss: 0.1719	Val loss: 0.2988	Val R2: 0.6908
Epoch 50	Train loss: 0.1690	Val loss: 0.3571	Val R2: 0.6341

```
import torch
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error
```

```
# --- tryb ewaluacji ---
model.eval()
```

```
# --- zbieramy predykcje ---
y_true_list = []
y_pred_list = []
```

```
with torch.no_grad():
    for Xb, yb in val_loader:
        Xb = Xb.to(device)
        preds = model(Xb)
        y_true_list.append(yb.cpu().numpy())
        y_pred_list.append(preds.cpu().numpy())
```

```
# --- składamy w jeden array ---
y_true_scaled = np.vstack(y_true_list)
y_pred_scaled = np.vstack(y_pred_list)
```

```
# --- odwrotna transformacja ---
y_true = scaler_y.inverse_transform(y_true_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled)
```

```
# --- metryki ---
r2 = r2_score(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
```

```
print(f"R²: {r2:.4f}")
print(f"MSE: {mse:.6f}")
```

```
R²: 0.6547
MSE: 0.000286
```

```

import matplotlib.pyplot as plt
import numpy as np

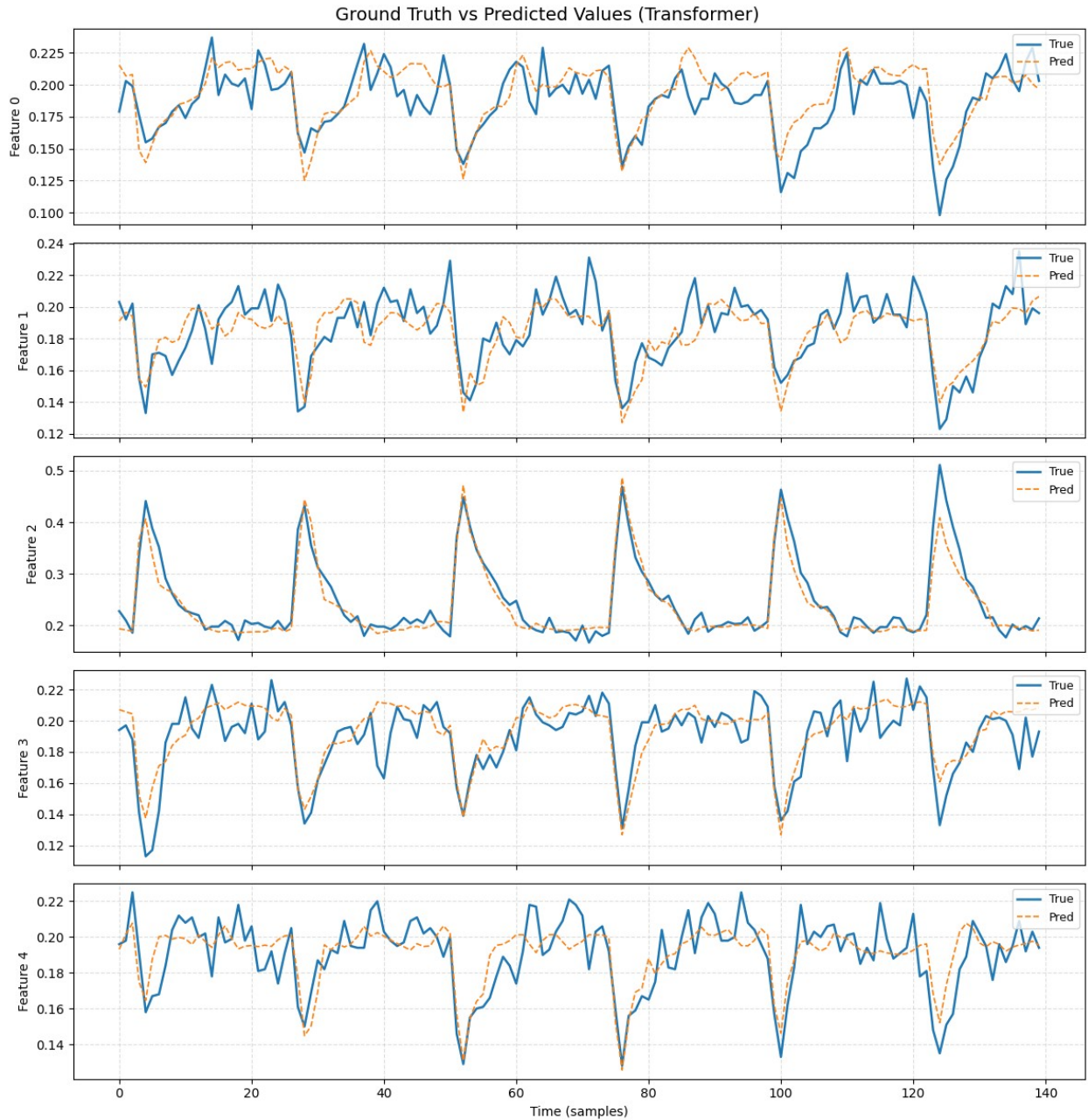
# Assume y_true and y_pred are numpy arrays of shape (n_samples, 5)
n_features = y_true.shape[1]
time = np.arange(len(y_true)) # x-axis index (could be timestamps if
you have them)

fig, axes = plt.subplots(n_features, 1, figsize=(12, 2.5 *
n_features), sharex=True)

for i in range(n_features):
    ax = axes[i]
    ax.plot(time, y_true[:, i], label="True", lw=1.8)
    ax.plot(time, y_pred[:, i], label="Pred", lw=1.2, linestyle='--')
    ax.set_ylabel(f"Feature {i}")
    ax.grid(True, linestyle='--', alpha=0.4)
    ax.legend(loc="upper right", fontsize=9)

axes[-1].set_xlabel("Time (samples)")
fig.suptitle("Ground Truth vs Predicted Values (Transformer)",
fontsize=14, y=0.98)
plt.tight_layout()
plt.show()

```



3.3.3 TODO

Test 6-8 configurations for various:

- window sizes
- embedding dimensions
- numbers of attention heads
- numbers of transformers layers
- feed-forward network size (dimensionality of the internal dense layer inside each Transformer block)

Give summary results in a table. Include parameters and regression scores. Add selected plots (e.g. for the best and worst results).

```
class TimeSeriesTransformer(nn.Module):
    def __init__(self, input_dim, d_model, n_heads, n_layers, ff_dim,
dropout=0.1):
        super().__init__()

        # Project input features (e.g. 5 → 64)
        self.input_proj = nn.Linear(input_dim, d_model)

        # Positional encoding
        self.pos_encoding = self._generate_pos_encoding(500, d_model)

        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=n_heads,
            dim_feedforward=ff_dim,
            dropout=dropout,
            batch_first=True,
            activation='gelu'
        )
        self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=n_layers)

        # Regression head
        self.head = nn.Sequential(
            nn.LayerNorm(d_model),
            nn.Linear(d_model, 64),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(64, input_dim)
        )

    def _generate_pos_encoding(self, max_len, d_model):
        """Precompute sinusoidal positional encodings"""
        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-
torch.log(torch.tensor(10000.0)) / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        return pe

    def forward(self, x):
        seq_len = x.size(1)
        x = self.input_proj(x) +
self.pos_encoding[:, :seq_len, :].to(x.device)
```

```

        x = self.transformer(x)
        x = x.mean(dim=1)
        # x = x.mean(dim=1) # Global average pooling over sequence
        return self.head(x)

def plot_results(y_true, y_pred, title):
    n_features = y_true.shape[1]
    time = np.arange(len(y_true))

    fig, axes = plt.subplots(n_features, 1, figsize=(12, 2.5 *
n_features), sharex=True)

    for i in range(n_features):
        ax = axes[i]
        ax.plot(time, y_true[:, i], label="True", lw=1.8)
        ax.plot(time, y_pred[:, i], label="Pred", lw=1.2,
linestyle="--")
        ax.set_ylabel(f"Feature {i}")
        ax.grid(True, linestyle='--', alpha=0.4)
        ax.legend(loc="upper right", fontsize=9)
    axes[-1].set_xlabel("Time (samples)")
    fig.suptitle(title, fontsize=14, y=0.98)
    plt.tight_layout()
    plt.show()

# d_model = 64          # embedding dimension
# num_heads = 4         # number of attention heads
# num_layers = 4        # number of transformer encoder layers
# ff_dim = 128          # feed-forward hidden dimension
# dropout = 0.1        # dropout rate
# batch_size = 32
# epochs = 50
# lr = 1e-3

# --- parameters ---
n_agents = 1000
n_locations = 5
center = 2
steps_per_day = 24
n_days = 30
T = steps_per_day * n_days

def create_window_dataset(window_size):
    X, y = [], []
    for i in range(T - window_size):
        X.append(df.iloc[i:i+window_size].values)
        y.append(df.iloc[i:i+window_size].values)
    return np.array(X), np.array(y)

config_ts = [

```

```

    {"name": "C1", "window": 24, "d_model": 32, "heads": 2, "layers":
2, "ff": 64},
    {"name": "C2", "window": 50, "d_model": 64, "heads": 4, "layers":
4, "ff": 128},
    {"name": "C3", "window": 12, "d_model": 128, "heads": 8, "layers":
4, "ff": 256},
    {"name": "C4", "window": 4, "d_model": 64, "heads": 8, "layers":
6, "ff": 256},
    {"name": "C5", "window": 32, "d_model": 128, "heads": 4, "layers":
2, "ff": 256},
    {"name": "C6", "window": 40, "d_model": 128, "heads": 8, "layers":
6, "ff": 64},
]

```

```

results = []

```

```

for cfg in config_ts:
    print(f"\n--- Running config: {cfg} ---")

    X, y = create_window_dataset(cfg["window"])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, shuffle=False
    )

    scaler_x = StandardScaler()
    scaler_y = StandardScaler()

    X_train_2d = X_train.reshape(-1, X_train.shape[-1])
    scaler_X.fit(X_train_2d)
    scaler_y.fit(y_train)

    X_train_scaled =
scaler_X.transform(X_train_2d).reshape(X_train.shape)
    X_test_scaled = scaler_X.transform(X_test.reshape(-1,
X_test.shape[-1])).reshape(X_test.shape)

    y_train_scaled = scaler_y.transform(y_train)
    y_test_scaled = scaler_y.transform(y_test)

    device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')

    X_train = torch.tensor(X_train_scaled, dtype=torch.float32)
    y_train = torch.tensor(y_train_scaled, dtype=torch.float32)
    X_val = torch.tensor(X_test_scaled, dtype=torch.float32)
    y_val = torch.tensor(y_test_scaled, dtype=torch.float32)

    train_loader = DataLoader(TensorDataset(X_train, y_train),
batch_size=batch_size, shuffle=True)

```

```

val_loader = DataLoader(TensorDataset(X_val, y_val),
batch_size=batch_size, shuffle=False)

model = TimeSeriesTransformer(
    input_dim=X_train.shape[2],
    d_model=cfg["d_model"],
    n_heads=cfg["heads"],
    n_layers=cfg["layers"],
    ff_dim=cfg["ff"],
    dropout=0.1
).to(device)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=lr)

for epoch in range(20):
    model.train()
    train_loss = 0
    for Xb, yb in train_loader:
        Xb, yb = Xb.to(device), yb.to(device)
        optimizer.zero_grad()
        preds = model(Xb)
        loss = criterion(preds, yb)
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * Xb.size(0)

    model.eval()
    val_loss, val_r2 = 0, 0
    with torch.no_grad():
        for Xb, yb in val_loader:
            Xb, yb = Xb.to(device), yb.to(device)
            preds = model(Xb).squeeze()
            val_loss += criterion(preds, yb).item() * Xb.size(0)
            val_r2 += r2_score(yb.cpu().numpy(),
preds.cpu().numpy()) * Xb.size(0)

    train_loss /= len(train_loader.dataset)
    val_loss /= len(val_loader.dataset)
    val_r2 /= len(val_loader.dataset)

    print(f"Epoch {epoch+1:02d} | Train loss: {train_loss:.4f} | "
          f"Val loss: {val_loss:.4f} | Val R2: {val_r2:.4f}")

model.eval()

y_true_list = []
y_pred_list = []

```

```

with torch.no_grad():
    for Xb, yb in val_loader:
        Xb = Xb.to(device)
        preds = model(Xb)
        y_true_list.append(yb.cpu().numpy())
        y_pred_list.append(preds.cpu().numpy())

y_true_scaled = np.vstack(y_true_list)
y_pred_scaled = np.vstack(y_pred_list)

y_true = scaler_y.inverse_transform(y_true_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled)

plot_results(y_true, y_pred, cfg["name"])
print(y_true.min(), y_true.max())
print(y_pred.min(), y_pred.max())

r2 = r2_score(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
mae = np.mean(np.abs(y_true - y_pred))

results.append({
    "window": cfg["window"],
    "d_model": cfg["d_model"],
    "heads": cfg["heads"],
    "layers": cfg["layers"],
    "ff_dim": cfg["ff"],
    "MSE": mse,
    "RMSE": rmse,
    "MAE": mae,
    "R2": r2
})

```

Output hidden; open in <https://colab.research.google.com> to view.

```

import pandas as pd

df_results = pd.DataFrame(results)
df_results = df_results.sort_values("R2",
ascending=False).reset_index(drop=True)

df_results

{"summary": "{\n  \"name\": \"df_results\",\n  \"rows\": 6,\n  \"fields\": [\n    {\n      \"column\": \"window\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 17,\n        \"min\": 4,\n        \"max\": 50,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          50,\n          12\n        ],\n        \"semantic_type\": \"\",

```



```
print("Best config:\n", df_results.loc[best_idx])
print("\nWorst config:\n", df_results.loc[worst_idx])
```

Best config:

```

window      32.000000
d_model     128.000000
heads       4.000000
layers      2.000000
ff_dim      256.000000
MSE         0.000253
RMSE        0.015895
MAE         0.012007
R2          0.684273
Name: 0, dtype: float64
```

Worst config:

```

window      12.000000
d_model     128.000000
heads       8.000000
layers      4.000000
ff_dim      256.000000
MSE         0.000548
RMSE        0.023413
MAE         0.014170
R2          0.543382
Name: 5, dtype: float64
```

```
def plot_results(y_true, y_pred, title):
    n_features = y_true.shape[1]
    time = np.arange(len(y_true))

    fig, axes = plt.subplots(n_features, 1, figsize=(12, 2.5 *
n_features), sharex=True)

    for i in range(n_features):
        ax = axes[i]
        ax.plot(time, y_true[:, i], label="True", lw=1.8)
        ax.plot(time, y_pred[:, i], label="Pred", lw=1.2,
linestyle="--")
        ax.set_ylabel(f"Feature {i}")
        ax.grid(True, linestyle='--', alpha=0.4)
        ax.legend(loc="upper right", fontsize=9)
    axes[-1].set_xlabel("Time (samples)")
    fig.suptitle(title, fontsize=14, y=0.98)
    plt.tight_layout()
    plt.show()
```

Tasks

Write a short report containing the following items. Use plots and tables. Make comments. Deliver as a PDF file.

- Section 3.1.3 Perform sentiment classification using IMDB dataset (35%)
 - Prepare 8 configurations (including various preprocessing parameters).
 - Report results in the table summarizing parameter values and classification results (report accuracy, precision, recall, F1).
 - Display a few reviews from the dataset and print classification result
 - Use Tensorflow or Pytorch library
- Section 3.2.4 Follow the specification (35%)
- Section 3.3.3 Follow the specification (20%)
- Answer the questions (10%)
 - a. What is the complexity of attention mechanism with respect to n - the sequence lengths
 - b. Is the attention map global or local (pertaining to the sequence).
 - c. Which parameters are learned during training the Transformers models for NLP?
 - d. What is the role of positional encoding?
 - e. Which output data resulting from Transformer Block can be used for classification or regression